

Containerization & DevOps Lab

Laboratory Portfolio

Name: SRIJATO DAS

SAP ID: 500119148

Enrollment No: R2142230488

Batch: B1 CCVT

Faculty Name: Dr. Prateek Raj Gautam

Contents

1	Containerization and DevOps Lab	6
1.0.1	University of Petroleum and Energy Studies	6
1.1	Student Profile	6
1.2	Table of Contents	6
1.3	Laboratory Overview	6
1.4	Experiment Summary Table	6
1.5	Detailed Experimental Summaries	7
1.5.1	Experiment 0: Windows Subsystem for Linux (WSL2) Configuration	7
1.5.2	Experiment 1: Comparative Analysis of Virtualization Methodologies	7
1.5.3	Experiment 2: Docker Fundamental Operations	7
1.5.4	Experiment 3: NGINX Deployment and Image Layer Optimization	7
1.5.5	Experiment 4: Docker Essential Runtimes	7
1.5.6	Experiment 5: Persistent Storage and Network Isolation	8
1.5.7	Experiment 6: Declarative Deployment with Docker Compose	8
1.5.8	Experiment 7: Automated CI/CD Pipeline Orchestration	8
1.5.9	Experiment 9: Infrastructure Automation via Ansible	8
1.5.10	Experiment 10: Static Application Security Testing (SAST)	8
1.5.11	Experiment 11: Basic Cluster Orchestration with Docker Swarm	8
1.5.12	Experiment 12: Advanced Orchestration via Kubernetes	8
1.6	Technical Specifications	8
1.7	Repository Organization	9
2	Experiment 0: Windows Subsystem for Linux (WSL) Configuration	10
2.1	Table of Contents	10
2.2	Objective	10
2.3	Prerequisites	10
2.4	Implementation Steps	10
2.4.1	Step 1: Install WSL and Ubuntu Distribution	10
2.4.2	Step 2: Restart the System	10
2.4.3	Step 3: Verify WSL Installation and Version	11
2.4.4	Step 4: Set Ubuntu as Default Distribution	11
2.4.5	Step 5: Configure WSL 2 as Global Default	11
2.4.6	Step 6: Install and Configure Git	11
2.5	Verification	12
2.5.1	Verify WSL2 Installation	12
2.5.2	Check Ubuntu Version	12
2.5.3	Update Package Manager	12
2.6	Troubleshooting	12
2.7	Performance Comparison	13
2.8	Conclusion	13
2.9	Additional Resources	13
3	Experiment 1: Comparison of Virtual Machines (VMs) and Containers	14
3.1	Table of Contents	14
3.2	Objective	14
3.3	Prerequisites	14
3.4	Virtual Machines (VMs) Implementation	14
3.4.1	Part A: VirtualBox and Vagrant Setup	14
3.4.2	Part A: Resource Monitoring in VM	18
3.4.3	Part A: Cleanup	20
3.5	Containers Implementation	20
3.5.1	Part B: Docker Installation in WSL	20
3.5.2	Part B: Nginx Container Deployment	21
3.5.3	Part B: Container Resource Monitoring	22
3.6	Resource Utilization Analysis	22

3.6.1	Comparison Metrics	22
3.6.2	Performance Characteristics	23
3.7	Comparison & Conclusion	23
3.7.1	Use Case Recommendations	23
3.7.2	Key Findings	24
3.7.3	Conclusion	24
3.8	Additional Resources	24
4	Experiment 2: Docker Installation, Configuration, and Running Images	25
4.1	Table of Contents	25
4.2	Objective	25
4.3	Procedure	25
4.3.1	Step 1: Pull Image	25
4.3.2	Step 2: Run Container with Port Mapping	25
4.3.3	Step 3: Verify Running Containers	26
4.3.4	Step 4: Stop and Remove Container	26
4.3.5	Step 5: Remove Image	26
4.4	Result	26
4.5	Overall Conclusion	26
4.6	Additional Resources	27
5	Experiment 3: Deploying NGINX Using Different Base Images and Comparing Image Layers	28
5.1	Table of Contents	28
5.2	Objectives	28
5.3	Prerequisites	28
5.4	Part 1: Official NGINX Image	28
5.4.1	Step 1: Pull Official NGINX Image	28
5.4.2	Step 2: Run Official NGINX Container	29
5.4.3	Step 3: Verify Official NGINX	29
5.4.4	Step 4: Inspect Official Image	29
5.5	Part 2: Ubuntu-Based NGINX	29
5.5.1	Step 1: Create Ubuntu-Based Dockerfile	30
5.5.2	Step 2: Build Ubuntu-Based Image	30
5.5.3	Step 3: Run Ubuntu-Based Container	30
5.5.4	Step 4: Verify Ubuntu-Based NGINX	30
5.6	Part 3: Alpine-Based NGINX	31
5.6.1	Step 1: Create Alpine-Based Dockerfile	31
5.6.2	Step 2: Build Alpine-Based Image	31
5.6.3	Step 3: Run Alpine-Based Container	31
5.6.4	Step 4: Verify Alpine-Based NGINX	31
5.7	Part 4: Image Analysis & Comparison	34
5.7.1	Step 1: Compare Image Sizes	34
5.7.2	Step 2: Inspect Image Layers	34
5.7.3	Step 3: Key Observations	35
5.8	Comparison Analysis	35
5.8.1	Performance Characteristics	35
5.8.2	Security Comparison	36
5.8.3	Use Case Recommendations	36
5.9	Cleanup	36
5.9.1	Stop and Remove Containers	36
5.9.2	Remove Images	36
5.10	Conclusion	36
5.11	Additional Resources	37
6	Experiment 4: Docker Essentials	38
6.1	Table of Contents	38
6.2	Objective	38

6.3	Prerequisites	38
6.4	Part A — Flask application (my-flask-app)	38
6.5	Part B — Optimization: Multi-stage build	47
6.6	Part C — Tagging and publishing to Docker Hub	49
6.7	Part D — Node.js App	50
6.8	Troubleshooting & Best Practices	52
6.9	Conclusion	52
6.10	Additional Resources	52
7	Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks	54
7.1	Table of Contents	54
7.2	Part 1: Docker Volumes - Persistent Data Storage	54
7.2.1	Lab 1: Understanding Data Persistence	54
7.2.2	Lab 2: Volume Types	55
7.2.3	Lab 3: Practical Volume Examples	57
7.2.4	Lab 4: Volume Management Commands	58
7.3	Part 2: Environment Variables	60
7.3.1	Lab 1: Setting Environment Variables	60
7.3.2	Lab 2: Environment Variables in Applications	61
7.4	Part 3: Docker Monitoring	62
7.4.1	Lab 1: Basic Monitoring Commands	62
7.4.2	Lab 2: docker top - Process Monitoring	63
7.4.3	Lab 3: docker logs - Application Logs	63
7.4.4	Lab 4: Container Inspection	63
7.4.5	Lab 5: Events Monitoring	67
7.5	Part 4: Docker Networks	68
7.5.1	Lab 1: Understanding Docker Network Types	68
7.5.2	Lab 2: Network Types Explained	68
7.6	Part 5: Complete Real-World Example	68
7.7	Quick Reference Cheatsheet	68
7.8	Practice Exercises	69
7.9	Key Takeaways	69
7.10	Additional Resources	69
8	Experiment 6: Docker Run vs. Docker Compose	70
8.1	Table of Contents	70
8.2	PART A – THEORY	70
8.2.1	1. Objective	70
8.2.2	2. Background Theory	70
8.2.3	3. Comparison: Docker Run vs. Docker Compose	70
8.3	PART B – PRACTICAL TASK	70
8.3.1	Task 1: Single Container Comparison	70
8.4	PART C – CONVERSION & BUILD-BASED TASKS	71
8.4.1	Task 2: Multi-Container Application (WordPress + MySQL)	71
8.5	PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE	74
8.5.1	Task 3: Replace Standard Image with Dockerfile	74
8.6	EXPERIMENT 6 B: Multi-Container Application involving Docker Swarm	80
8.6.1	1. Running with Docker Swarm	80
8.7	Key Learning Outcomes	83
8.8	Additional Resources	83
9	Experiment 7: Jenkins, GitHub, and Docker Hub	84
9.1	Table of Contents	84
9.2	1. Aim	84
9.3	2. Objectives	84
9.4	3. Theory	84
9.5	Part A: GitHub Repository Setup	84

9.5.1	5.1 Project Structure	84
9.5.2	5.3 Jenkinsfile (Pipeline Definition)	86
9.6	Part B: Jenkins Setup using Docker	86
9.6.1	6.1 Create Docker Compose File	86
9.7	Part E: Understanding Jenkins Pipeline Syntax	87
9.7.1	Key Terms	87
9.8	Observations & Results	87
9.9	Additional Resources	87
10	Experiment 9: Infrastructure Automation using Ansible	88
10.1	Table of Contents	88
10.2	1. Theory	88
10.2.1	What is Ansible?	88
10.3	PART A – PRACTICAL TASK	88
10.3.1	2. Ansible Installation on Windows (via WSL)	88
10.4	3. Ansible Demo with Docker Containers	88
10.4.1	Step 1: Create SSH Key Pair in WSL	88
10.4.2	Step 2: Create Dockerfile for SSH Server	91
10.4.3	Step 3: Build and Run	91
10.5	4. Multi-Container Ansible Exercise	91
10.5.1	Step 1: Start 4 Test Servers	91
10.5.2	Step 2: Run Playbook (playbook.yml)	92
10.6	5. The Need for Ansible	94
10.7	Conclusion	94
10.8	Additional Resources	94
11	Experiment 10: SonarQube — Static Code Analysis	95
11.1	Table of Contents	95
11.2	1. Theory	95
11.2.1	What is SonarQube?	95
11.2.2	Key Concepts	95
11.3	2. Lab Architecture	95
11.4	3. Hands-on Lab	95
11.4.1	Step 1: Start the SonarQube Server	95
11.4.2	Step 2: Generate Token	98
11.4.3	Step 2: Run the Scanner	99
11.4.4	Step 3: View Results	99
11.5	4. Integration & Best Practices	100
11.5.1	CI/CD Integration (Jenkins)	100
11.5.2	Best Practices	100
11.6	Additional Resources	100
12	Experiment 11: Orchestration using Docker Swarm	101
12.1	Table of Contents	101
12.2	1. Theory: Concept Continuation	101
12.2.1	What is Orchestration?	101
12.2.2	The Progression Path	101
12.3	2. Practical Tasks	101
12.3.1	Task 1: Initialize Docker Swarm	101
12.3.2	Task 2: Deploy as a Stack	102
12.3.3	Task 3: Scale the Application	102
12.3.4	Task 4: Test Self-Healing	102
12.4	3. Analysis: Compose vs Swarm	103
12.5	4. Important Observations	104
12.6	Additional Resources	104
13	Experiment 12: Container Orchestration using Kubernetes	105

13.1	Table of Contents	105
13.2	1. Objective	105
13.3	2. Theory: Why Kubernetes?	105
13.4	Core Concepts Mapping	105
13.5	3. Hands-On Lab	105
13.5.1	Task 1: Create a Deployment	105
13.5.2	Task 2: Expose the Deployment	106
13.5.3	Task 3: Scale the Deployment	106
13.5.4	Task 4: Test Self-Healing	106
13.6	5. Summary of Commands (Cheat Sheet)	109
13.7	6. Conclusion	110
13.8	Additional Resources	110

1 Containerization and DevOps Lab

1.0.1 University of Petroleum and Energy Studies

School of Computer Science | Cloud Computing & Virtualization Technology



1.1 Student Profile

- **Name:** Srijato Das
- **Sap Id:** 500119148
- **Enrollment No:** R2142230488
- **Batch:** B1 CCVT
- **Subject:** Containerization and DevOps

1.2 Table of Contents

1. [Laboratory Overview](#)
2. [Experiment Summary Table](#)
3. [Detailed Experimental Summaries](#)
 - [Exp 0: WSL Configuration](#)
 - [Exp 1: Virtualization Methodologies](#)
 - [Exp 2: Docker Primaries](#)
 - [Exp 3: Layered Architecture](#)
 - [Exp 4: Docker Essentials](#)
 - [Exp 5: Persistence & Networks](#)
 - [Exp 6: Declarative Orchestration](#)
 - [Exp 7: CI/CD Pipelines](#)
 - [Exp 9: Ansible Automation](#)
 - [Exp 10: Static Code Analysis](#)
 - [Exp 11: Docker Swarm](#)
 - [Exp 12: Kubernetes Orchestration](#)
4. [Technical Specifications](#)
5. [Repository Organization](#)

1.3 Laboratory Overview

This technical portfolio documents a comprehensive series of laboratory experiments focused on the architecture, deployment, and management of cloud-native infrastructure. The curriculum provides an end-to-end perspective on the DevOps lifecycle, moving from local environment virtualization to advanced cluster orchestration and automated security analysis.

1.4 Experiment Summary Table

ID	Module Title	Primary Focus	Technical Stack	Documentation
0	WSL Configuration	Environment Setup	WSL2, Ubuntu, Git	View
1	Virtualization Analysis	VMs vs. Containers	Vagrant, VirtualBox	View

ID	Module Title	Primary Focus	Technical Stack	Documentation
2	Docker Primaries	Image Lifecycle	Docker CLI, Nginx	View
3	Layered Architecture	Image Optimization	Alpine, UnionFS	View
4	Docker Essentials	Runtime Config	Env Vars, Mapping	View
5	Data Persistence	Volume Management	Docker Volumes, Bridge	View
6	Declarative Deployment	Service Orchestration	Docker Compose, YAML	View
7	Pipeline Automation	CI/CD Integration	Jenkins, GitHub, Hub	View
9	Configuration Management	Infrastructure as Code	Ansible, SSH, YAML	View
10	Static Analysis	Security & Quality	SonarQube, Maven	View
11	Cluster Management	Swarm Orchestration	Docker Swarm, Mesh	View
12	Advanced Orchestration	Kubernetes API	K8s, Pods, Services	View

1.5 Detailed Experimental Summaries

1.5.1 Experiment 0: Windows Subsystem for Linux (WSL2) Configuration

- **Objective:** Establishment of a high-performance, POSIX-compliant virtualization layer on Windows.
- **Key Tasks:** Enabling Virtual Machine Platform, installing Ubuntu 22.04 LTS, and configuring Git for cross-platform version control.
- **Outcome:** A unified development environment bridge between Windows and Linux kernels.
- **Manual:** [Lab 0 Manual](#)

1.5.2 Experiment 1: Comparative Analysis of Virtualization Methodologies

- **Objective:** Empirical study comparing Hypervisor-based virtualization (VMs) with OS-level containerization.
- **Key Tasks:** Deploying Ubuntu via Vagrant/VirtualBox and comparing its boot-time/resource usage against Docker containers.
- **Outcome:** Deep understanding of hardware abstraction vs. kernel sharing.
- **Manual:** [Lab 1 Manual](#)

1.5.3 Experiment 2: Docker Fundamental Operations

- **Objective:** Mastering the container lifecycle through the Docker CLI.
- **Key Tasks:** Pulling images from Docker Hub, running detached containers, and managing ephemeral container states.
- **Outcome:** Proficiency in image acquisition and container runtime management.
- **Manual:** [Lab 2 Manual](#)

1.5.4 Experiment 3: NGINX Deployment and Image Layer Optimization

- **Objective:** Optimization of storage footprint through layered architecture analysis.
- **Key Tasks:** Building custom NGINX images using Ubuntu and Alpine bases to compare UnionFS layer metadata.
- **Outcome:** Implementation of lightweight, security-hardened images (Alpine reduction of ~90%).
- **Manual:** [Lab 3 Manual](#)

1.5.5 Experiment 4: Docker Essential Runtimes

- **Objective:** Advanced runtime configuration and application containerization.
- **Key Tasks:** Environment variable injection, custom port mapping, and executing interactive bash sessions within containers.
- **Outcome:** Flexible deployment patterns for multi-environment application logic.
- **Manual:** [Lab 4 Manual](#)

1.5.6 Experiment 5: Persistent Storage and Network Isolation

- **Objective:** Implementing non-volatile storage and isolated inter-container networking.
- **Key Tasks:** Creating Docker Volumes for data persistence and custom bridge networks for secure service-to-service communication.
- **Outcome:** Resilient data management and hardened network boundaries.
- **Manual:** [Lab 5 Manual](#)

1.5.7 Experiment 6: Declarative Deployment with Docker Compose

- **Objective:** Transitioning from imperative commands to declarative multi-service stacks.
- **Key Tasks:** Orchestrating a WordPress/MySQL stack using a single `docker-compose.yml` file.
- **Outcome:** Repeatable, version-controlled multi-container infrastructure.
- **Manual:** [Lab 6 Manual](#)

1.5.8 Experiment 7: Automated CI/CD Pipeline Orchestration

- **Objective:** Construction of an end-to-end automated software delivery pipeline.
- **Key Tasks:** Integrating Jenkins with GitHub webhooks to trigger automated Docker builds and pushes to Docker Hub.
- **Outcome:** Zero-touch deployment workflow from commit to production registry.
- **Manual:** [Lab 7 Manual](#)

1.5.9 Experiment 9: Infrastructure Automation via Ansible

- **Objective:** Agentless configuration management for large-scale infrastructure.
- **Key Tasks:** Distributing SSH keys and executing idempotent YAML playbooks to standardize remote server states.
- **Outcome:** Eliminating configuration drift through automated system provisioning.
- **Manual:** [Lab 9 Manual](#)

1.5.10 Experiment 10: Static Application Security Testing (SAST)

- **Objective:** Integrating automated quality gates into the DevOps lifecycle.
- **Key Tasks:** Using SonarQube to analyze code for vulnerabilities, bugs, and technical debt.
- **Outcome:** Enhanced software reliability and security posture prior to runtime.
- **Manual:** [Lab 10 Manual](#)

1.5.11 Experiment 11: Basic Cluster Orchestration with Docker Swarm

- **Objective:** Transitioning to distributed cluster management and high availability.
- **Key Tasks:** Initializing a Swarm manager, scaling services horizontally, and implementing self-healing via health checks.
- **Outcome:** Resilient, load-balanced application delivery across multi-node clusters.
- **Manual:** [Lab 11 Manual](#)

1.5.12 Experiment 12: Advanced Orchestration via Kubernetes

- **Objective:** Leveraging the industry-standard Kubernetes API for workload management.
 - **Key Tasks:** Configuration of Pods, Deployments, and NodePort Services to manage complex containerized ecosystems.
 - **Outcome:** Enterprise-grade orchestration with advanced scheduling and lifecycle management.
 - **Manual:** [Lab 12 Manual](#)
-

1.6 Technical Specifications

- **Operating System:** Windows 11 Enterprise / Ubuntu 22.04 LTS (WSL2)
- **Container Engine:** Docker v24.0.7
- **IaC Tool:** Ansible v2.10.8

- **CI/CD Platform:** Jenkins LTS
 - **Security Analysis:** SonarQube Community Edition
 - **Orchestration:** Kubernetes v1.29.0 / Docker Swarm
-

1.7 Repository Organization

- **/Lab:** Technical documentation and experimental procedures.
- **/Asset:** Empirical evidence, diagrams, and terminal logs.
- **/Project_Assignment:** Specialized network and security implementations.
- **PDF Generation Guide:** Manual for converting the portfolio to a submission-ready PDF.

2 Experiment 0: Windows Subsystem for Linux (WSL) Configuration

2.1 Table of Contents

1. [Objective](#)
 2. [Prerequisites](#)
 3. [Implementation Steps](#)
 4. [Verification](#)
 5. [Troubleshooting](#)
 6. [Conclusion](#)
-

2.2 Objective

Establish a functional Linux development environment on a Windows host using Windows Subsystem for Linux (WSL2) and install Git for version control. This setup provides the necessary foundation for running DevOps tools, containerization engines (Docker), and Linux-based development environments natively on Windows.

2.3 Prerequisites

- **OS:** Windows 10 (Build 19041+) or Windows 11
 - **Hardware:** Virtualization capability enabled in BIOS
 - **Permissions:** Administrator access to PowerShell
 - **Disk Space:** Minimum 5GB free storage
-

2.4 Implementation Steps

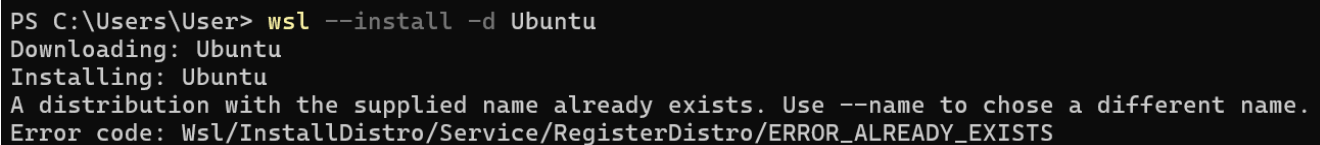
2.4.1 Step 1: Install WSL and Ubuntu Distribution

Open **PowerShell as Administrator** and execute the installation command.

Command:

```
wsl --install --distribution Ubuntu
```

Important Note: A system restart is required after this command completes. Save all work before proceeding.

A screenshot of a PowerShell terminal window. The prompt is 'PS C:\Users\User>'. The user has entered the command 'wsl --install -d Ubuntu'. The output shows 'Downloading: Ubuntu', 'Installing: Ubuntu', and then an error message: 'A distribution with the supplied name already exists. Use --name to chose a different name. Error code: Wsl/InstallDistro/Service/RegisterDistro/ERROR_ALREADY_EXISTS'.

```
PS C:\Users\User> wsl --install -d Ubuntu
Downloading: Ubuntu
Installing: Ubuntu
A distribution with the supplied name already exists. Use --name to chose a different name.
Error code: Wsl/InstallDistro/Service/RegisterDistro/ERROR_ALREADY_EXISTS
```

Figure 1: WSL Installation

2.4.2 Step 2: Restart the System

After running the installation command, restart your computer to allow Windows to initialize the WSL2 kernel and virtual machine platform.

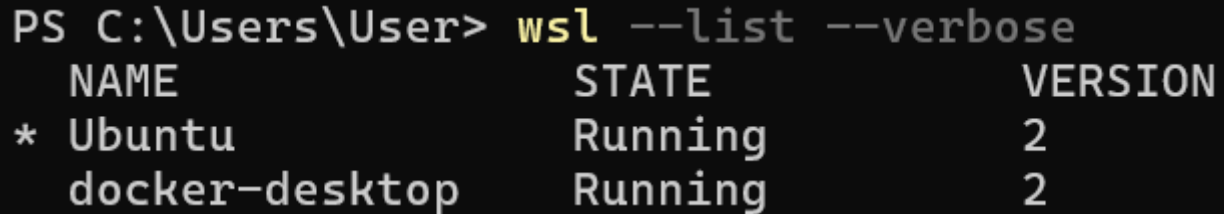
[Restart-Computer](#)

2.4.3 Step 3: Verify WSL Installation and Version

Once the system has rebooted, verify the installation.

Command:

```
wsl --list --verbose
```



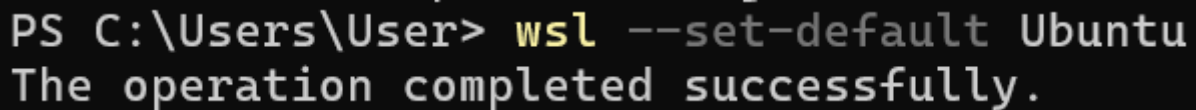
```
PS C:\Users\User> wsl --list --verbose
  NAME                STATE      VERSION
*  Ubuntu              Running    2
   docker-desktop      Running    2
```

Figure 2: WSL List Verbose

2.4.4 Step 4: Set Ubuntu as Default Distribution

Command:

```
wsl --set-default Ubuntu
```



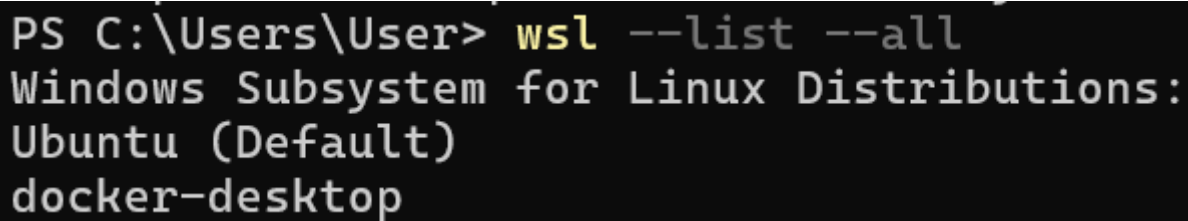
```
PS C:\Users\User> wsl --set-default Ubuntu
The operation completed successfully.
```

Figure 3: Set Default Distribution

2.4.5 Step 5: Configure WSL 2 as Global Default

Command:

```
wsl --set-default-version 2
```



```
PS C:\Users\User> wsl --list --all
Windows Subsystem for Linux Distributions:
Ubuntu (Default)
docker-desktop
```

Figure 4: WSL2 Default Version

2.4.6 Step 6: Install and Configure Git

2.4.6.1 6.1: Update Package Manager

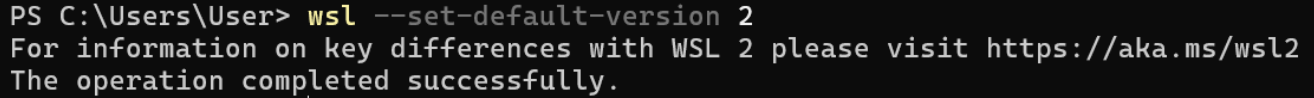
```
sudo apt update
```


2.4.6.2 6.2: Install Git

```
sudo apt install -y git
```

2.4.6.3 6.3: Verify Git Installation

```
git --version
```



```
PS C:\Users\User> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
```

Figure 5: Git Installation Verification

2.4.6.4 6.4: Configure Git User Information

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2.4.6.5 6.5: Configure Git for WSL Integration

```
git config --global core.autocrlf input
```

2.5 Verification

2.5.1 Verify WSL2 Installation

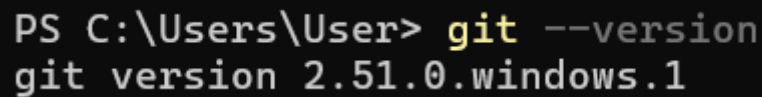
```
wsl --list --verbose
```

2.5.2 Check Ubuntu Version

```
lsb_release -a
```

2.5.3 Update Package Manager

```
sudo apt update && sudo apt upgrade -y
```



```
PS C:\Users\User> git --version
git version 2.51.0.windows.1
```

Figure 6: WSL Verification and Update

2.6 Troubleshooting

Issue	Solution
“Virtual Machine Platform not enabled”	Enable it in Windows Features: Settings > Apps > Apps & features > Optional features > Virtual Machine Platform
“Virtualization not enabled in BIOS”	Restart computer, enter BIOS (Del/F2), enable VT-x (Intel) or AMD-V (AMD)
WSL1 instead of WSL2	Run <code>wsl --set-default-version 2</code> and convert existing instances

Issue	Solution
Slow WSL2 performance on network drives	Store projects in WSL's native file system (/home/username/) instead of /mnt/c/
"Distribution not found"	Run <code>wsl --install --distribution Ubuntu</code> again or download from Microsoft Store

2.7 Performance Comparison

Feature	WSL 1	WSL 2
File System Performance	Native Windows	Virtual Machine
Linux Compatibility	~80%	100%
Docker Support	Limited	Full
Memory Usage	Low	Medium
Boot Time	Instant	~3-5 seconds

Recommendation: WSL2 is strongly preferred for DevOps and containerization work.

2.8 Conclusion

You have successfully established a fully functional Linux development environment on Windows. This environment is now ready for version control, containerization (Docker), Kubernetes, and advanced DevOps tooling.

Next Steps: Proceed to Experiment 1 to install Docker and containerization tools.

2.9 Additional Resources

- [WSL Official Documentation](#)
- [Git Documentation](#)
- [Ubuntu on WSL Guide](#)

3 Experiment 1: Comparison of Virtual Machines (VMs) and Containers

3.1 Table of Contents

1. [Objective](#)
 2. [Prerequisites](#)
 3. [Virtual Machines \(VMs\) Implementation](#)
 4. [Containers Implementation](#)
 5. [Resource Utilization Analysis](#)
 6. [Comparison & Conclusion](#)
-

3.2 Objective

Understand and practically compare the conceptual and architectural differences between Virtual Machines (VMs) and Containers through hands-on deployment. This experiment evaluates performance metrics, resource utilization, and use case suitability of both virtualization approaches.

Expected Outcomes:

- Successfully deploy a full virtual machine using VirtualBox and Vagrant
 - Deploy containerized applications using Docker in WSL
 - Compare boot times, memory usage, and resource overhead
 - Understand when to use VMs vs. Containers in DevOps workflows
 - Analyze isolation levels and performance characteristics
-

3.3 Prerequisites

- **Windows OS:** Windows 10 (Build 19041+) or Windows 11
 - **WSL2:** Fully configured with Ubuntu (from Experiment 0)
 - **Git:** Installed and configured (from Experiment 0)
 - **VirtualBox:** Version 7.0 or later
 - **Vagrant:** Version 2.3 or later
 - **Docker:** Will be installed during this experiment
 - **Disk Space:** Minimum 20GB free storage
 - **RAM:** Minimum 8GB (16GB recommended)
-

3.4 Virtual Machines (VMs) Implementation

3.4.1 Part A: VirtualBox and Vagrant Setup

Virtual Machines provide full OS-level isolation by emulating complete hardware through a hypervisor.

3.4.1.1 Step 1: Install VirtualBox Download and install Oracle VirtualBox from [virtualbox.org](https://www.virtualbox.org).

Installation Verification:

```
VBoxManage --version
```

Expected Output:

```
7.0.14r161095
```

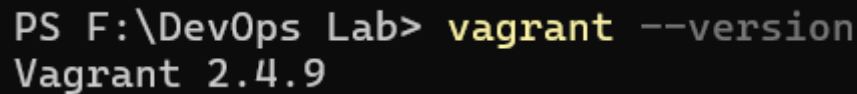
3.4.1.2 Step 2: Install Vagrant Download and install Vagrant from vagrantup.com.

Installation Verification:

```
vagrant --version
```

Expected Output:

Vagrant 2.4.1



```
PS F:\DevOps Lab> vagrant --version
Vagrant 2.4.9
```

Figure 7: Vagrant Installation

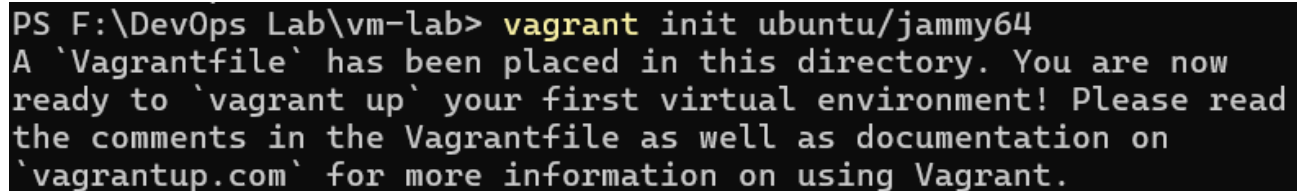
3.4.1.3 Step 3: Create VM Project Directory Organize your project structure for VM deployment:

```
mkdir vm-lab
cd vm-lab
```

3.4.1.4 Step 4: Initialize Vagrant with Ubuntu Initialize a Vagrant environment using the official Ubuntu Jammy 64-bit box:

```
vagrant init ubuntu/jammy64
```

This command creates a Vagrantfile with default configurations for the Ubuntu image.



```
PS F:\DevOps Lab\vm-lab> vagrant init ubuntu/jammy64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

Figure 8: Vagrant Init Configuration

3.4.1.5 Step 5: Deploy the Virtual Machine Launch the virtual machine (this may take 5-10 minutes for the first download):

```
vagrant up
```

Expected Output:

```
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'ubuntu/jammy64' could not be found locally...
==> default: Adding box 'ubuntu/jammy64' (v20240101.0.0) from Vagrant Cloud...
...
==> default: Machine booted and ready for use.
```

3.4.1.6 Step 6: Access the Virtual Machine Connect to the running VM via SSH:

```
vagrant ssh
```

3.4.1.7 Step 1: Update Package Manager

```
sudo apt update
```

```

PS F:\DevOps Lab\vm-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'ubuntu/jammy64' could not be found. Attempting to find and install...
default: Box Provider: virtualbox
default: Box Version: >= 0
==> default: Loading metadata for box 'ubuntu/jammy64'
default: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
==> default: Adding box 'ubuntu/jammy64' (v20241002.0.0) for provider: virtualbox
default: Downloading: https://vagrantcloud.com/ubuntu/boxes/jammy64/versions/20241002.0.0/providers/virtualbox/unknown/vagrant.box
default:
==> default: Successfully added box 'ubuntu/jammy64' (v20241002.0.0) for 'virtualbox'!
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Setting the name of the VM: vm-lab_default_1770736122749_61905
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
default: Adapter 1: nat
==> default: Forwarding ports...
default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
default: SSH address: 127.0.0.1:2222
default: SSH username: vagrant
default: SSH auth method: private key
default: Warning: Connection reset. Retrying...
default: Warning: Connection aborted. Retrying...
default:
default: Vagrant insecure key detected. Vagrant will automatically replace
default: this with a newly generated keypair for better security.
default:
default: Inserting generated public key within guest...
default: Removing insecure key from the guest if it's present...
default: Key inserted! Disconnecting and reconnecting using new SSH key...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
default: The guest additions on this VM do not match the installed version of
default: VirtualBox! In most cases this is fine, but in rare cases it can
default: prevent things such as shared folders from working properly. If you see
default: shared folder errors, please make sure the guest additions within the
default: virtual machine match the version of VirtualBox you have installed on
default: your host and reload your VM.
default:
default: Guest Additions Version: 6.0.0 r127566
default: VirtualBox Version: 7.2
==> default: Mounting shared folders...
default: F:\DevOps Lab\vm-lab => /vagrant

```

Figure 9: Vagrant Up Deployment

```
vagrant@ubuntu-jammy: ~  
PS F:\DevOps Lab\vm-lab> vagrant ssh  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-164-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Tue Feb 10 15:10:48 UTC 2026  
  
System load:            0.14  
Usage of /:             4.0% of 38.70GB  
Memory usage:          21%  
Swap usage:            0%  
Processes:             107  
Users logged in:       0  
IPv4 address for enp0s3: 10.0.2.15  
IPv6 address for enp0s3: fd17:625c:f037:2:28:41ff:fee4:55a4  
  
Expanded Security Maintenance for Applications is not enabled.  
  
1 update can be applied immediately.  
1 of these updates is a standard security update.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
New release '24.04.3 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
vagrant@ubuntu-jammy:~$ sudo apt update  
sudo apt install -y nginx  
sudo systemctl start nginx  
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease  
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
```

Figure 10: Vagrant SSH Access

3.4.1.8 Step 2: Install Nginx

```
sudo apt install -y nginx
```

Expected Output:

```
Reading package lists... Done
Setting up nginx (1.18.0-6ubuntu14.4) ...
Processing triggers for man-db (2.10.2-1) ...
```

3.4.1.9 Step 3: Start Nginx Service

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

3.4.1.10 Step 4: Verify Nginx is Running

```
sudo systemctl status nginx
```

Expected Output:

- nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/lib/systemd/nginx.service; enabled; vendor preset: enabled)
Active: active (running) since Mon 2026-02-10 10:30:25 UTC; 5s ago
-

3.4.2 Part A: Resource Monitoring in VM

Test the Nginx server:

```
curl localhost
```

Expected Output:

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</html>
```

Monitor VM resource consumption while running:

```
# Check RAM usage
free -h
```

```
# Check CPU usage
top -b -n 1 | head -20
```

```
# Check disk usage
df -h
```

Expected Output (Memory):

Type	Total	Used	Free
Memory	1.9 GiB	340 MiB	1.2 GiB

```

vagrant@ubuntu-jammy:~$ curl localhost
vagrant halt
vagrant destroy
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Figure 11: Nginx Running in VM

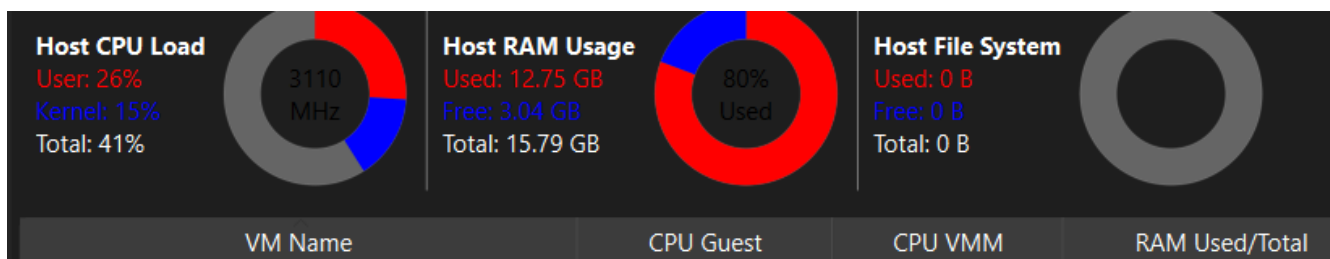
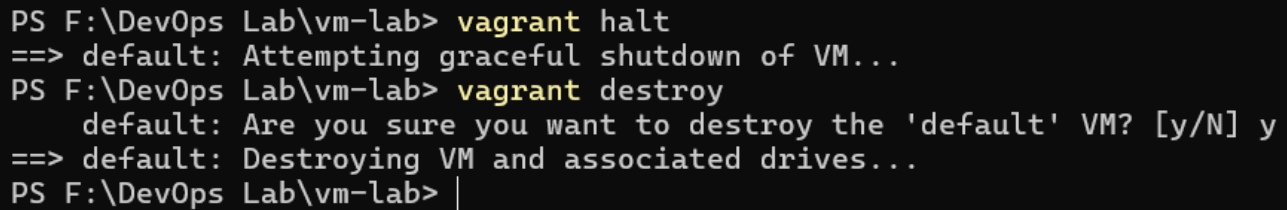


Figure 12: VM Resource Monitoring

3.4.3 Part A: Cleanup

Exit the SSH session and terminate the VM:

```
exit
vagrant halt
vagrant destroy -f
```



```
PS F:\DevOps Lab\vm-lab> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS F:\DevOps Lab\vm-lab> vagrant destroy
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS F:\DevOps Lab\vm-lab> |
```

Figure 13: VM Destruction

3.5 Containers Implementation

3.5.1 Part B: Docker Installation in WSL

Containers provide lightweight virtualization by sharing the host OS kernel while isolating applications.

3.5.1.1 Step 1: Update WSL Ubuntu Open WSL Ubuntu terminal and update packages:

```
sudo apt update && sudo apt upgrade -y
```

3.5.1.2 Step 2: Install Docker Engine Install Docker and its dependencies:

```
sudo apt install -y docker.io docker-compose
```

Expected Output:

```
Setting up docker.io (24.0.7-1~ubuntu.22.04~jammy) ...
Processing triggers for man-db (2.10.2-1) ...
```

3.5.1.3 Step 3: Enable Docker Service Start the Docker daemon and enable it for auto-start:

```
sudo systemctl start docker
sudo systemctl enable docker
```

3.5.1.4 Step 4: Configure User Permissions Add your user to the docker group to avoid using sudo for every command:

```
sudo usermod -aG docker $USER
newgrp docker
```

3.5.1.5 Step 5: Verify Docker Installation Check Docker version and test functionality:

```
docker --version
docker run hello-world
```

Expected Output:

```
Docker version 24.0.7, build afdd53b
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
root@LAPTOP-080561KA: /mr x + v
root@LAPTOP-080561KA:/mnt/ff/DevOps Lab/vm-lab# docker pull ubuntu
docker run -d -p 8080:80 --name nginx-container nginx
curl localhost:8080
Using default tag: latest
latest: Pulling from library/ubuntu
a3629ac5b9f4: Pull complete
Digest: sha256:cd1dbae651b3080c3686ecf4e3c4220f026b521fb76978881737d24f2008282b2b
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
0c8d5a45c8d: Pull complete
46b73a120e8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
f73408a233fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e26e0056d313208
Status: Downloaded newer image for nginx:latest
b06caa68ab0ealeb6f7add445fb72ee80b3225ba85d80536b81f083918764044
docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime create failed: runc create failed: invalid rootfs: stat /run/desktop-contai
nerd/daemon/io.containerd.runtime.v2.task/moby/b06caa68ab0ealeb6f7add445fb72ee80b3225ba85d80536b81f083918764044/E:\DockerData\vfs\dir\3f19df2cb2641eb15da422a7c6a7c221488ff8b6509ad4917c879c
f6da122393: no such file or directory

Run 'docker run --help' for more information
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Figure 14: Docker Installation Verification

3.5.2 Part B: Nginx Container Deployment

3.5.2.1 Step 1: Pull Nginx Image

Download the official Nginx Docker image:

```
docker pull nginx:latest
```

Expected Output:

```
latest: Pulling from library/nginx
a803e7c4b030: Pull complete
82efea989f9c: Pull complete
...
Status: Downloaded newer image for nginx:latest
```

3.5.2.2 Step 2: Run Nginx Container

Launch an Nginx container with port mapping:

```
docker run -d -p 8080:80 --name nginx-container nginx:latest
```

Parameters Explained:

- d: Run in detached mode (background)
- p 8080:80: Map host port 8080 to container port 80
- --name nginx-container: Assign a friendly name
- nginx:latest: Image to deploy

Expected Output:

```
7a8c9d2e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9
```

3.5.2.3 Step 3: Verify Container is Running

List running containers:

```
docker ps
```

Expected Output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-official -p 8080:80 nginx
8869e95b98804e047496e4b9e61be5a3babfe101f6f12c0ea2979abed317919c
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |

```

Figure 15: Running Ngnix

```

7a8c9d2e4f5a    nginx:latest    "/docker-entrypoint...."    10 seconds ago    Up 9 seconds    0.0.0.0:8080-
>80/tcp    nginx-container

```

☐	Name	Tag	Image ID	Created	Size
☐	ubuntu	latest	493218ed0f40	28 days ago	78.12 MB
☒	nginx	latest	5cdef4ac3335	6 days ago	160.82 MB

Figure 16: Docker Container Running

3.5.2.4 Step 4: Test Web Server Access

Access the Nginx container through the mapped port:

```
curl localhost:8080
```

Expected Output:

```

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</html>

```

3.5.3 Part B: Container Resource Monitoring

Monitor container resource usage in real-time:

```
docker stats nginx-container
```

Expected Output:

Container ID	Name	CPU %	Mem Usage / Limit	Mem %	Net I/O
7a8c9d2e4f5a	nginx-container	0.02%	12.5MiB / 2GiB	0.61%	1.2MB / 890kB

☐	Name	Tag	Image ID	Created	Size
☐	ubuntu	latest	493218ed0f40	28 days ago	78.12 MB
☒	nginx	latest	5cdef4ac3335	6 days ago	160.82 MB

Metric	Virtual Machine	Container	Winner
Boot Time	30-60 seconds	< 1 second	Container
Memory Usage (Idle)	512MB - 1GB	5-15MB	Container
Memory Usage (Running)	800MB - 1.2GB	25-50MB	Container
CPU Overhead	5-10%	< 1%	Container
Disk Space	2-3GB per VM	100-300MB per image	Container
Startup Time	30-60 seconds	100-500ms	Container
OS Isolation	Complete	Process-level	VM
Deployment Speed	Minutes	Seconds	Container

3.6.2 Performance Characteristics

3.6.2.1 Virtual Machines Advantages:

- Complete isolation and security
- Support for different OS types
- Suitable for legacy applications
- Full resource dedication possible

Disadvantages:

- High memory overhead
- Slow boot times
- Larger disk footprint
- Resource-intensive for simple workloads

3.6.2.2 Containers Advantages:

- Minimal resource overhead
- Extremely fast startup
- Portable across platforms
- Ideal for microservices
- Easy orchestration and scaling

Disadvantages:

- Process-level isolation (not OS-level)
- All containers share host OS kernel
- Less suitable for GUI applications
- Requires container orchestration for complex deployments

3.7 Comparison & Conclusion

3.7.1 Use Case Recommendations

Use Virtual Machines when:

- You need complete OS isolation
- Running multiple different operating systems
- Legacy application compatibility required
- Full security boundary needed
- Compliance regulations demand OS-level isolation

Use Containers when:

- Building microservices architectures
- Rapid deployment and scaling needed
- Resource optimization is critical
- Consistent environments across dev/prod
- DevOps and CI/CD pipelines

3.7.2 Key Findings

1. **Resource Efficiency:** Containers consume 50-100x less memory than VMs for similar workloads
2. **Boot Time:** Containers start in milliseconds vs. minutes for VMs
3. **Scalability:** Containers allow running dozens of instances where VMs allow only a few
4. **Management:** Container orchestration (Docker, Kubernetes) simplifies deployment and scaling
5. **Architecture:** Modern DevOps favors containers for application deployment while VMs remain valuable for infrastructure management

3.7.3 Conclusion

This experiment demonstrates that **containers are the preferred choice for modern DevOps workflows** due to their efficiency, speed, and scalability. However, Virtual Machines remain essential for scenarios requiring complete OS isolation or specific operating system requirements.

For most cloud-native and microservices-based applications, **Docker containerization is the industry standard**, supported by orchestration platforms like Kubernetes.

3.8 Additional Resources

- [Docker Official Documentation](#)
- [VirtualBox Manual](#)
- [Vagrant Documentation](#)
- [Docker Hub](#)
- [Kubernetes Official Site](#)
- [Cloud Native Computing Foundation](#)

4 Experiment 2: Docker Installation, Configuration, and Running Images

4.1 Table of Contents

1. [Objective](#)
 2. [Procedure](#)
 3. [Result](#)
 4. [Conclusion](#)
-

4.2 Objective

- Pull Docker images
 - Run containers
 - Manage container lifecycle
-

4.3 Procedure

4.3.1 Step 1: Pull Image

Pull the Nginx image from Docker Hub:

```
docker pull nginx
```

```
PS F:\DevOps Lab\ Docker Labs> wsl
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
0c8d55a45c0d: Pull complete
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Figure 18: Pull Docker Image

4.3.2 Step 2: Run Container with Port Mapping

Run a container with port mapping (host port 8080 to container port 80):

```
docker run -d -p 8080:80 nginx
```

```
PS F:\DevOps Lab\ Docker Labs> wsl
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d -p 8080:80 nginx
4eb5b40b422c112880ad23c049850bb9f6f701c6f8044345b5687a6517b01fa8
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d -p 8080:80 nginx
e6a764d971a23154b6d625363995e98ac2b10c6a2a33bcb5efc6ebc3cdf8dc02
```

4.3.3 Step 3: Verify Running Containers

List all running containers:

```
docker ps
```

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
e6a764d971a2   nginx    "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   determined_pike
```

Figure 19: Verify Running Containers

4.3.4 Step 4: Stop and Remove Container

Stop the running container and remove it:

```
docker stop <container_id>
docker rm <container_id>
```

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker stop e6a
e6a
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker rm e6a
e6a
```

Figure 20: Stop Container

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |
```

Figure 21: Remove Container

4.3.5 Step 5: Remove Image

Remove the Docker image:

```
docker rmi nginx
```

4.4 Result

Docker images were successfully pulled, containers executed, and lifecycle commands performed.

4.5 Overall Conclusion

This lab demonstrated containerization using Docker, highlighting container management basics. Containers are better suited for rapid deployment and microservices, while VMs provide stronger isolation.

☐	Name	Tag	Image ID	Created	Size	Actions
☐	nginx	latest	341bf0f3ce6c	9 days ago	239.91 MB	  

Figure 22: Remove Image

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker stop e6a
e6a
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker rm e6a
e6a
```

Figure 23: Result Summary

4.6 Additional Resources

- [Docker Official Documentation](#)
- [Docker Hub Registry](#)

5 Experiment 3: Deploying NGINX Using Different Base Images and Comparing Image Layers

5.1 Table of Contents

1. [Objectives](#)
 2. [Prerequisites](#)
 3. [Part 1: Official NGINX Image](#)
 4. [Part 2: Ubuntu-Based NGINX](#)
 5. [Part 3: Alpine-Based NGINX](#)
 6. [Part 4: Image Analysis & Comparison](#)
 7. [Conclusion](#)
 8. [Additional Resources](#)
-

5.2 Objectives

After completing this lab, you will achieve:

- Deploy NGINX using Official, Ubuntu-based, and Alpine-based images
 - Understand Docker image layers and size optimization
 - Compare performance, security, and use-cases for each base image
 - Analyze image history and layer construction
 - Optimize Docker images for production deployment
-

5.3 Prerequisites

- **Docker:** Installed and running in WSL (from Experiment 1)
 - **Docker CLI:** Version 20.10 or later
 - **Basic Knowledge:** Docker run, Dockerfile, port mapping
 - **Terminal:** WSL Ubuntu or Windows Terminal with WSL
 - **Disk Space:** Minimum 5GB available
-

5.4 Part 1: Official NGINX Image

The recommended approach for production uses the pre-optimized official image from Docker Hub.

5.4.1 Step 1: Pull Official NGINX Image

Pull the latest official NGINX image:

```
docker pull nginx:latest
```

Expected Output:

```
latest: Pulling from library/nginx
a803e7c4b030: Pull complete
82efea989f9c: Pull complete
...
Status: Downloaded newer image for nginx:latest
```

5.4.2 Step 2: Run Official NGINX Container

Run the container with port mapping:

```
docker run -d --name nginx-official -p 8080:80 nginx:latest
```

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-official -p 8080:80 nginx
8869e95b98804e047496e4b9e61be5a3babfe101f6f12c0ea2979abed317919c
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |
```

Figure 24: Run Official NGINX Image

5.4.3 Step 3: Verify Official NGINX

Test the running container:

```
curl http://localhost:8080
docker ps | grep nginx-official
```

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker images nginx
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    5cdef4ac3335   2 weeks ago   161MB
nginx         <none>    4af177a024eb   5 weeks ago   161MB
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |
```

Figure 25: Official NGINX Running

5.4.4 Step 4: Inspect Official Image

View image details and size:

```
docker images nginx:latest
docker inspect nginx:latest | grep -i "size\|created\|architecture"
```

5.5 Part 2: Ubuntu-Based NGINX

Create a custom NGINX image using Ubuntu as the base image.

5.5.1 Step 1: Create Ubuntu-Based Dockerfile

Create a new directory and Dockerfile:

```
mkdir -p ~/docker-images/nginx-ubuntu
cd ~/docker-images/nginx-ubuntu
```

Dockerfile Content:

```
FROM ubuntu:22.04

# Update package lists
RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

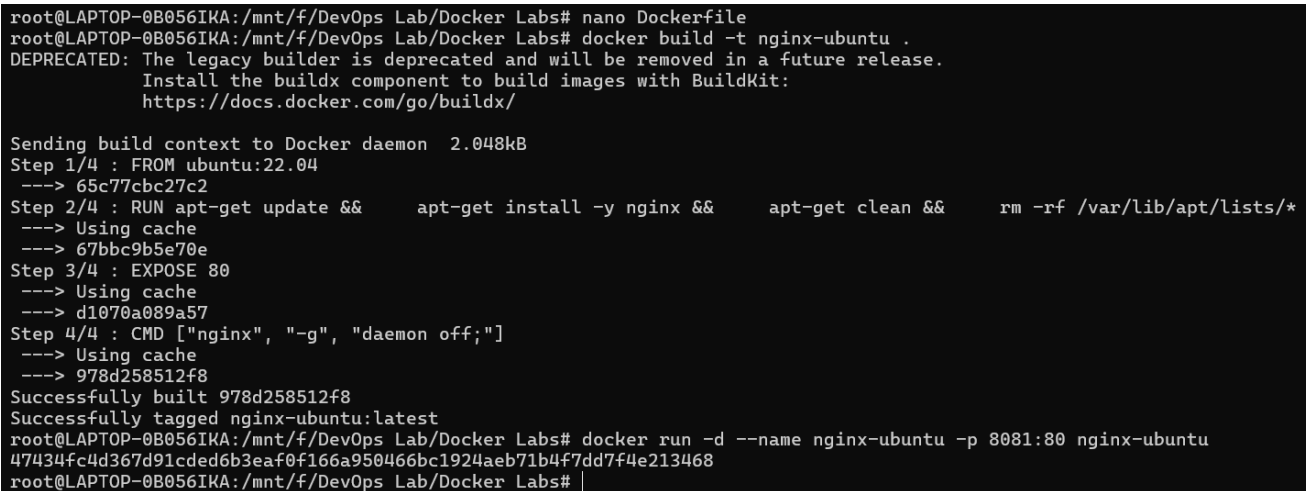
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

5.5.2 Step 2: Build Ubuntu-Based Image

Build the custom image:

```
docker build -t nginx-ubuntu:latest .
```



```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# nano Dockerfile
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker build -t nginx-ubuntu .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:22.04
--> 65c77cbc27c2
Step 2/4 : RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 67bbc9b5e70e
Step 3/4 : EXPOSE 80
--> Using cache
--> d1070a089a57
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
--> Using cache
--> 978d258512f8
Successfully built 978d258512f8
Successfully tagged nginx-ubuntu:latest
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
47434fc4d367d91cded6b3eaf0f166a950466bc1924aeb71b4f7dd7f4e213468
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs#
```

Figure 26: Build Ubuntu Image

5.5.3 Step 3: Run Ubuntu-Based Container

Run the Ubuntu-based container:

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu:latest
```

5.5.4 Step 4: Verify Ubuntu-Based NGINX

Test the container:

```
curl http://localhost:8081
docker ps | grep nginx-ubuntu
```

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker images nginx-ubuntu
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
nginx-ubuntu    latest      978d258512f8   13 days ago   134MB
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs#
```

Figure 27: Ubuntu NGINX Image Verification

5.6 Part 3: Alpine-Based NGINX

Create an optimized NGINX image using Alpine Linux.

5.6.1 Step 1: Create Alpine-Based Dockerfile

Create Alpine Dockerfile:

```
mkdir -p ~/docker-images/nginx-alpine
cd ~/docker-images/nginx-alpine
```

Dockerfile Content:

```
FROM alpine:latest

# Install NGINX
RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

5.6.2 Step 2: Build Alpine-Based Image

Build the Alpine image:

```
docker build -t nginx-alpine:latest .
```

5.6.3 Step 3: Run Alpine-Based Container

Run the Alpine container:

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine:latest
```

5.6.4 Step 4: Verify Alpine-Based NGINX

Test the container:

```
curl http://localhost:8082
docker ps | grep nginx-alpine
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker build -t nginx-alpine .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM alpine:latest
latest: Pulling from library/alpine
589002ba0eae: Pulling fs layer
589002ba0eae: Verifying Checksum
589002ba0eae: Download complete
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
----> a40c03cbb81c
Step 2/4 : RUN apk add --no-cache nginx
----> Running in 3424cb0c1cac
(1/2) Installing pcre2 (10.47-r0)
(2/2) Installing nginx (1.28.2-r0)
     Executing nginx-1.28.2-r0.pre-install
     Executing nginx-1.28.2-r0.post-install
Executing busybox-1.37.0-r30.trigger
OK: 9.9 MiB in 18 packages
----> Removed intermediate container 3424cb0c1cac
----> 6c82f9406eba
Step 3/4 : EXPOSE 80
----> Running in 5299636ff03f
----> Removed intermediate container 5299636ff03f
----> 06b335f73530
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
----> Running in ba020dbdc681
----> Removed intermediate container ba020dbdc681
----> 21e9c176b0e4
Successfully built 21e9c176b0e4
Successfully tagged nginx-alpine:latest
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
6f8e97a40b4e223a0ae2074c1e711560b9d6769ca49364adf7ff2c5a8739181c
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |

```

Figure 28: Create Alpine Dockerfile

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker build -t nginx-alpine .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM alpine:latest
latest: Pulling from library/alpine
589002ba0eae: Pulling fs layer
589002ba0eae: Verifying Checksum
589002ba0eae: Download complete
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
----> a40c03cbb81c
Step 2/4 : RUN apk add --no-cache nginx
----> Running in 3424cb0c1cac
(1/2) Installing pcre2 (10.47-r0)
(2/2) Installing nginx (1.28.2-r0)
     Executing nginx-1.28.2-r0.pre-install
     Executing nginx-1.28.2-r0.post-install
Executing busybox-1.37.0-r30.trigger
OK: 9.9 MiB in 18 packages
----> Removed intermediate container 3424cb0c1cac
----> 6c82f9406eba
Step 3/4 : EXPOSE 80
----> Running in 5299636ff03f
----> Removed intermediate container 5299636ff03f
----> 06b335f73530
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
----> Running in ba020dbdc681
----> Removed intermediate container ba020dbdc681
----> 21e9c176b0e4
Successfully built 21e9c176b0e4
Successfully tagged nginx-alpine:latest
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
6f8e97a40b4e223a0ae2074c1e711560b9d6769ca49364adf7ff2c5a8739181c
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |

```

Figure 29: Build Alpine Image

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker run -d --name nginx-official -p 8080:80 nginx
8869e95b98804e047496e4b9e61be5a3babfe101f6f12c0ea2979abed317919c
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker images nginx-alpine
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx-alpine         latest             21e9c176b0e4       About a minute ago  10.4MB
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |

```

5.7 Part 4: Image Analysis & Comparison

5.7.1 Step 1: Compare Image Sizes

View all three images side by side:

```
docker images | grep -E "nginx-official|nginx-ubuntu|nginx-alpine|^REPOSITORY"
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker images | grep nginx
nginx-alpine         latest             21e9c176b0e4       About a minute ago  10.4MB
nginx-ubuntu         latest             978d258512f8       13 days ago        134MB
nginx                latest             5cdef4ac3335       2 weeks ago        161MB
nginx                <none>             4af177a024eb       5 weeks ago        161MB
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# |

```

Figure 30: Image Size Comparison

5.7.2 Step 2: Inspect Image Layers

View the layer history for each image:

Official Image Layers:

```
docker history nginx:latest
```

Ubuntu Image Layers:

```
docker history nginx-ubuntu:latest
```

Alpine Image Layers:

```
docker history nginx-alpine:latest
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs# docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
5cde44ac3335   2 weeks ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     STOPSIGNAL SIGQUIT                             0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     EXPOSE map[80/tcp:{}]                          0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENTRYPOINT ["/docker-entrypoint.sh"]           0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 30-tune-worker-processes.sh /docker-ent... 4.62kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 20-envsubst-on-templates.sh /docker-ent... 3.02kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 15-local-resolvers.envsh /docker-entryp... 389B      buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 10-listen-on-ipv6-by-default.sh /docker... 2.12kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY docker-entrypoint.sh / # buildkit         1.62kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     RUN /bin/sh -c set -x && groupadd --syst...    82.2MB    buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV DYNPKG_RELEASE=1~trixie                    0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV PKG_RELEASE=1~trixie                       0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV ACME_VERSION=0.3.1                         0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_RELEASE=1~trixie                       0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_VERSION=0.9.5                         0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NGINX_VERSION=1.29.5                       0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 78.6MB    debuerreotype 0.17
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
978d258512f8   13 days ago     /bin/sh -c #(nop) CMD ["nginx" "-g" "daemon... 0B
d1070a089a57   13 days ago     /bin/sh -c #(nop) EXPOSE 80                    0B
67bb9b5e70e    13 days ago     /bin/sh -c apt-get update && apt-get ins...    55.7MB
65c77cbc27c2   6 weeks ago     /bin/sh -c #(nop) CMD ["/bin/bash"]            0B
<missing>      6 weeks ago     /bin/sh -c #(nop) ADD file:b499000226bd9a7c5... 77.9MB
<missing>      6 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      6 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      6 weeks ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      6 weeks ago     /bin/sh -c #(nop) ARG RELEASE                  0B
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
21e9c176b0e4   2 minutes ago   /bin/sh -c #(nop) CMD ["nginx" "-g" "daemon... 0B
06b335f73530   2 minutes ago   /bin/sh -c #(nop) EXPOSE 80                    0B
6c82f9406eba   2 minutes ago   /bin/sh -c apk add --no-cache nginx            2MB
a40c03cbb81c   3 weeks ago     CMD ["/bin/sh"]                                0B        buildkit.dockerfile.v0
<missing>      3 weeks ago     ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /... 8.44MB    buildkit.dockerfile.v0
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs#

```

Figure 31: Docker Image Layers Analysis

5.7.3 Step 3: Key Observations

Official Image (nginx:latest):

- Pre-optimized and well-maintained
- Based on Debian (slim variant)
- Best for production deployments
- Good balance between size and functionality

Ubuntu-Based (nginx-ubuntu):

- Large image size due to full Ubuntu base
- Many filesystem layers
- Excellent for development with full tools
- Larger attack surface

Alpine-Based (nginx-alpine):

- Extremely small image size
- Minimal dependencies and security surface
- Fastest deployment and pull times
- Limited debugging tools

5.8 Comparison Analysis

5.8.1 Performance Characteristics

Metric	Official	Ubuntu	Alpine
Pull Time	~20 seconds	~45 seconds	~5 seconds
Container Startup	~1 second	~2 seconds	~0.5 seconds
Memory Usage (Idle)	~12 MB	~18 MB	~4 MB
Disk Space	187 MB	320 MB	28 MB

5.8.2 Security Comparison

Aspect	Official	Ubuntu	Alpine
CVE Surface	Medium	High	Very Low
Attack Surface	Optimized	Large	Minimal
Updates	Regular	Regular	Less frequent
Use Case	Production	Development	Edge/Lightweight

5.8.3 Use Case Recommendations

Use Official Image when:

- Building production applications
- Need proven, maintained image
- Want balance between size and features
- Security and updates are critical

Use Ubuntu-Based when:

- Building development/testing environments
- Need full OS utilities and tools
- Debugging requires shell access
- Building complex multi-service containers

Use Alpine-Based when:

- Minimizing image size is critical
- Deploying to resource-constrained environments
- Building microservices and edge applications
- Fast deployment and pull speed required

5.9 Cleanup

5.9.1 Stop and Remove Containers

```
docker stop nginx-official nginx-ubuntu nginx-alpine
docker rm nginx-official nginx-ubuntu nginx-alpine
```

5.9.2 Remove Images

```
docker rmi nginx:latest nginx-ubuntu:latest nginx-alpine:latest
```

5.10 Conclusion

This experiment demonstrated:

- Deployment of NGINX using three different base images
- Analysis of Docker image layers and their impact on size

- Performance and security trade-offs of each approach
- Real-world use cases for different base image selections
- Image optimization techniques for production deployment

Key Takeaways:

1. Alpine provides significant size advantages (85-90% smaller)
2. Official images offer best production practices
3. Ubuntu provides development convenience at cost of size
4. Image selection impacts security, performance, and deployment

Next Steps: Proceed to Experiment 4 for Docker Essentials and advanced container practices.

5.11 Additional Resources

- [Docker Official Images](#)
- [Alpine Linux Official](#)
- [Dockerfile Best Practices](#)
- [Docker Image Layers Guide](#)

6 Experiment 4: Docker Essentials

6.1 Table of Contents

1. [Objective](#)
 2. [Prerequisites](#)
 3. [Part A — Flask application](#)
 4. [Part B — Optimization: Multi-stage build](#)
 5. [Part C — Tagging and publishing to Docker Hub](#)
 6. [Part D — Node.js App](#)
 7. [Troubleshooting & Best Practices](#)
 8. [Conclusion](#)
 9. [Additional Resources](#)
-

6.2 Objective

Master core Docker skills including containerizing applications with Dockerfile, optimizing builds with .dockerignore, implementing multi-stage builds, and publishing verified images to Docker Hub. This lab uses a simple Python Flask app and a small Node.js example to demonstrate patterns.

6.3 Prerequisites

- WSL2 / Ubuntu or Linux environment
 - Docker Engine installed and running
 - Docker Hub account (for publishing)
 - Basic familiarity with Python and Node.js
-

6.4 Part A — Flask application (my-flask-app)

A lightweight Flask app is created, containerized, optimized, and published.

Project structure:

```
my-flask-app/  
├─ app.py  
├─ requirements.txt  
├─ Dockerfile  
├─ Dockerfile.multistage  
└─ .dockerignore
```

- 1) Create project directory and files

```
mkdir my-flask-app  
cd my-flask-app
```

- 2) app.py (simple web server)

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# nano app.py
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# nano requirements.txt
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# nano Dockerfile
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# nano .dockerignore
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Build image from Dockerfile
docker build -t my-flask-app .

# Check built images
docker images
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
--> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in a43933db408e
Collecting Flask==2.3.3
  Downloading flask-2.3.3-py3-none-any.whl (96 kB)
    _____ 96.1/96.1 kB 1.4 MB/s eta 0:00:00
Collecting Jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
    _____ 134.9/134.9 kB 3.2 MB/s eta 0:00:00
Collecting Werkzeug>=2.3.7
  Downloading werkzeug-3.1.6-py3-none-any.whl (225 kB)
    _____ 225.2/225.2 kB 16.4 MB/s eta 0:00:00
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
    _____ 98.2/98.2 kB 75.4 MB/s eta 0:00:00
Collecting itsdangerous>=2.1.2
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Collecting blinker>=1.6.2
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)

```

Figure 32: Project directory created

```
@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
Collecting blinker>=1.6.2
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting zipp>=3.2.0
  Downloading zipp-3.23.0-py3-none-any.whl (10 kB)
Collecting MarkupSafe>=2.0
  Downloading MarkupSafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Installing collected packages: zipp, MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, importlib-metadata, Flask
Successfully installed Flask-2.3.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.6 blinker-1.9.0 click-8.1.8 importlib-metadata-8.7.1 itsdangerous-2.2.0 zipp-3.23.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
--> Removed intermediate container a43933db408e
--> 30dafdf759bf
Step 5/7 : COPY app.py .
--> 6db757594587
Step 6/7 : EXPOSE 5000
--> Running in 7addc8167182
--> Removed intermediate container 7addc8167182
--> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
--> Running in 24bd87abd3a4
--> Removed intermediate container 24bd87abd3a4
--> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged my-flask-app:latest
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	latest	74b04e3e595c	Less than a second ago	132MB
nginx-alpine	latest	21e9c176b0e4	18 minutes ago	10.4MB
nginx-ubuntu	latest	978d258512f8	13 days ago	134MB
python-app	v1	b3195e66a200	2 weeks ago	206MB
nginx	latest	5cdef4ac3335	2 weeks ago	161MB
alpine	latest	a40c03cbb81c	3 weeks ago	8.44MB
mcr.microsoft.com/openjdk/jdk	17-ubuntu	da5a772aac7	4 weeks ago	443MB
ubuntu	latest	493218ed0f40	5 weeks ago	78.1MB
nginx	<none>	4af177a024eb	5 weeks ago	161MB
ubuntu	22.04	65c77cbc27c2	6 weeks ago	77.9MB
python	3.9-slim	085da638e1b8	3 months ago	122MB
hello-world	latest	1b44b5a3e06a	6 months ago	10.1kB

Figure 33: app.py created

3) requirements.txt

Flask==2.3.3

4) .dockerignore — keep the build context small

```
__pycache__/
*.pyc
.env
.venv
.vscode/
.idea/
.git/
.DS_Store
logs/
```

5) Dockerfile (simple)

```
# Use Python base image
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Tag with version number
docker build -t my-flask-app:1.0 .

# Tag with multiple tags
docker build -t my-flask-app:latest -t my-flask-app:1.0 .

# Tag with custom registry
docker build -t username/my-flask-app:1.0 .

# Tag existing image
docker tag my-flask-app:latest my-flask-app:v1.0
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 30dafdf759bf
Step 5/7 : COPY app.py .
----> Using cache
----> 6db757594587
Step 6/7 : EXPOSE 5000
----> Using cache
----> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged my-flask-app:1.0
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
----> Using cache

```

Figure 34: requirements

```

Step 3/7 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 30dafdf759bf
Step 5/7 : COPY app.py .
----> Using cache
----> 6db757594587
Step 6/7 : EXPOSE 5000
----> Using cache
----> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged my-flask-app:latest
Successfully tagged my-flask-app:1.0
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 30dafdf759bf
Step 5/7 : COPY app.py .
----> Using cache
----> 6db757594587
Step 6/7 : EXPOSE 5000
----> Using cache
----> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged username/my-flask-app:1.0
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# |

```

Figure 35: dockerignore

EXPOSE 5000

CMD ["python", "app.py"]

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # List all images
docker images

# Show image history
docker history my-flask-app

# Inspect image details
docker inspect my-flask-app
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	1.0	74b04e3e595c	3 minutes ago	132MB
my-flask-app	latest	74b04e3e595c	3 minutes ago	132MB
my-flask-app	v1.0	74b04e3e595c	3 minutes ago	132MB
username/my-flask-app	1.0	74b04e3e595c	3 minutes ago	132MB
nginx-alpine	latest	21e9c176b0e4	21 minutes ago	10.4MB
nginx-ubuntu	latest	978d258512f8	13 days ago	134MB
python-app	v1	b3195e66a200	2 weeks ago	206MB
nginx	latest	5cdef4ac3335	2 weeks ago	161MB
alpine	latest	a40c03cbb81c	3 weeks ago	8.44MB
mcr.microsoft.com/openjdk/jdk	17-ubuntu	da5a772aac7	4 weeks ago	443MB
ubuntu	latest	493218ed0f40	5 weeks ago	78.1MB
nginx	<none>	4af177a024eb	5 weeks ago	161MB
ubuntu	22.04	65c77c7c27c2	6 weeks ago	77.9MB
python	3.9-slim	085da638e1b8	3 months ago	122MB
hello-world	latest	1b44b5a3e06a	6 months ago	10.1kB

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
74b04e3e595c	3 minutes ago	/bin/sh -c #(nop) CMD ["python" "app.py"]	0B	
07082cf2afb9	3 minutes ago	/bin/sh -c #(nop) EXPOSE 5000	0B	
6db757594587	3 minutes ago	/bin/sh -c #(nop) COPY file:444a3904a5c3d221...	228B	
30dafdf759bf	3 minutes ago	/bin/sh -c pip install --no-cache-dir -r req...	10.5MB	
345f87fccd8d	3 minutes ago	/bin/sh -c #(nop) COPY file:f095db6d544b2123...	13B	
8b3f3e41932e	2 weeks ago	/bin/sh -c #(nop) WORKDIR /app	0B	
085da638e1b8	3 months ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; for src in idle3 p...	36B	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	39.5MB	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8...	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PYTHON_VERSION=3.9.25	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3...	0B	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; apt-get update; a...	3.81MB	buildkit.dockerfile.v0
<missing>	3 months ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	4 months ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	78.6MB	debuerreotype 0.16

Figure 36: Dockerfile

6) Build and run image

- Build image: `docker build -t my-flask-app:latest .`
- Verify images: `docker images`
- Run container: `docker run -d -p 5000:5000 --name flask-container my-flask-app:latest`
- Test app: `curl http://localhost:5000`


```
[
  {
    "Id": "sha256:7ab4e3e595cbc8976db1ee0e5dc8c7da72f82644cd184aa080231257edd082",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0",
      "username/my-flask-app:1.0"
    ],
    "RepoDigests": [],
    "Parent": "sha256:07802cf2afb9ac770574222459f13db364d7a7d08ffacfa3517133dbcc26905",
    "Comment": "",
    "Created": "2026-02-20T13:16:26.275780719Z",
    "DockerVersion": "26.2.2",
    "Author": "",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=03F792829C04825C080DE09E9026995E310258568",
        "PYTHON_VERSION=3.9.20",
        "PYTHON_SHA256=80e07d7cf0c2f0cc0024321ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "Cmd": [
        "python",
        "app.py"
      ],
      "WorkingDir": "/app"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 132402301,
    "GraphDriver": {
      "Data": {
        "LowerDir": "/var/lib/docker/overlay2/351731ad78f5491c4d8ccffea4ac6ed92c4cf3819eca636d07cc292da009342a/diff:/var/lib/docker/overlay2/7ea53c5125c31bb010097a1c8977c194f1be59e8cd43a8794c145b4ae605c2b/diff:/var/lib/docker/overlay2/45d4db34557106b39f70u496847864df55f98b3179e3b5250164fbd/diff:/var/lib/docker/overlay2/821b408778bee90bc9471d129a9ea93f20ed4db5cd3bc4ba98685ede5045/diff:/var/lib/docker/overlay2/a9787936c11147536ed7a42c381f5bb0d7b202705ec9c52a2602609ac269f22/diff:/var/lib/docker/overlay2/2bcc344c645c04d0892d7924653d123f231169340b332d0e5b221091c37/diff:/var/lib/docker/overlay2/7a08ed083c12c3806ba612554adcd0bb4902b0650840471a5e36a3624a2a/diff",
        "MergedDir": "/var/lib/docker/overlay2/8ca129db0e98ecf6252f478c91a7be08129a3eedb1a0b486ccf720f536d3083/merged",
        "UpperDir": "/var/lib/docker/overlay2/8ca129db0e98ecf6252f478c91a7be08129a3eedb1a0b486ccf720f536d3083/diff",
        "WorkDir": "/var/lib/docker/overlay2/8ca129db0e98ecf6252f478c91a7be08129a3eedb1a0b486ccf720f536d3083/work"
      },
      "Name": "overlay2"
    },
    "RootFS": {
      "Type": "Layers",
      "Layers": [
        "sha256:d7c97cbef1fe7cae982649e9f55efe201212e8acca4b4d668c083b20e4e4fd4c",
        "sha256:4227758f8e472cb9e6f89f97ab0d864f6f6cd0d85edc06a27a9c93",
        "sha256:250992a89a831a8f26375a53f216f5fca35ecf392956f0d5eafaa15dfca",
        "sha256:c8f6b54339a8b0471408bd33d6d99dc7db2ed9cdcf7c533ba09efecd5f33f",
        "sha256:946dc9e92970408037e2096dd276d216620702060333aaddf7c31bae2",
        "sha256:c3865ee436419102e3ba716dc799b3ea5ab5804c7b301409309d99ba0c011",
        "sha256:168dd5bf75c5e6612db884dc9b86f40e0e0eb5a54d2ab92881d114c3a5e89f",
        "sha256:f836de23c04ad6ccf427a15c82ad312fbaf9a3bf0b71b1319d1b05c132b3e"
      ]
    },
    "Metadata": {
      "LastTagTime": "2026-02-20T13:10:16.143881609Z"
    }
  }
]
```

Figure 37: build output

```
root@LAPTOP-0B0561KA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Run container with port mapping
docker run -d -p 5000:5000 --name flask-container my-flask-app

# Test the application
curl http://localhost:5000

# View running containers
docker ps

# View container logs
docker logs flask-container
46e312884f2c182d4c850e489b893db3546fc8f2b2baa6a98675aa0674b3d54db
curl: (56) Recv failure: Connection reset by peer
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
46e312884f2c   my-flask-app   "python app.py"          Less than a second ago   Up Less than a second   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
6f8c074a0bb4e   nginx-alpine   "nginx -g 'daemon of..." 22 minutes ago   Up 22 minutes   0.0.0.0:8082->80/tcp, [::]:8082->80/tcp         nginx-alpine
47434fc4d367     nginx-ubuntu   "nginx -g 'daemon of..." 30 minutes ago   Up 30 minutes   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp         nginx-ubuntu
8869e95b0800     nginx          "/docker-entrypoint..." 38 minutes ago   Up 38 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp         nginx-official

* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.5:5000
Press CTRL+C to quit
root@LAPTOP-0B0561KA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app#
```

Figure 38: docker images list

```
root@LAPTOP-0B0561KA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Run container with port mapping
docker run -d -p 5000:5000 --name flask-container my-flask-app

# Test the application
curl http://localhost:5000

# View running containers
docker ps

# View container logs
docker logs flask-container
9f5ef95cb1beef5b97c747d3e58f4719a2897b915b35c042e884fa09c1beeb4
curl: (56) Recv failure: Connection reset by peer
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
9f5ef95cb1be   my-flask-app   "python app.py"          1 second ago   Up Less than a second   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
root@LAPTOP-0B0561KA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Stop container
docker stop flask-container

# Start stopped container
docker start flask-container

# Remove container
docker rm flask-container

# Remove container forcefully
docker rm -f flask-container
flask-container
flask-container
Error response from daemon: cannot remove container "flask-container": container is running: stop the container before removing or force remove
flask-container
```

Figure 39: container running

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# nano Dockerfile.multistage
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Build regular image
docker build -t flask-regular .

# Build multi-stage image
docker build -f Dockerfile.multistage -t flask-multistage .

# Compare sizes
docker images | grep flask-

# Expected output:
# flask-regular      ~250MB
# flask-multistage   ~150MB (40% smaller!)
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 30dafdf759bf
Step 5/7 : COPY app.py .
----> Using cache
----> 6db757594587
Step 6/7 : EXPOSE 5000
----> Using cache
----> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged flask-regular:latest
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/15 : FROM python:3.9-slim AS builder

```

```

Sending build context to Docker daemon 6.656kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 30dafdf759bf
Step 5/7 : COPY app.py .
----> Using cache
----> 6db757594587
Step 6/7 : EXPOSE 5000
----> Using cache
----> 07082cf2afb9
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 74b04e3e595c
Successfully built 74b04e3e595c
Successfully tagged flask-regular:latest
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 6.656kB
Step 1/15 : FROM python:3.9-slim AS builder
----> 085da638e1b8
Step 2/15 : WORKDIR /app
----> Using cache
----> 8b3f3e41932e
Step 3/15 : COPY requirements.txt .
----> Using cache
----> 345f87fccd8d
Step 4/15 : RUN python -m venv /opt/venv
----> Running in 8e8ad15a7593
----> Removed intermediate container 8e8ad15a7593
----> 13ad6aebc589
Step 5/15 : ENV PATH="/opt/venv/bin:$PATH"
----> Running in 8639d92e8739
----> Removed intermediate container 8639d92e8739
----> b2261f54dcfa
Step 6/15 : RUN pip install --no-cache-dir -r requirements.txt
----> Running in c0ca4c27f4ed
Collecting Flask==2.3.3

```

```

Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting zipp>=3.2.0
Downloading zipp-3.23.0-py3-none-any.whl (10 kB)
Collecting MarkupSafe>=2.0
Downloading MarkupSafe-3.0.3-cp39-cp39-manylinux2014_x86_64-manylinux_2_17_x86_64-manylinux_2_28_x86_64.whl (20 kB)
Installing collected packages: zipp, MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, importlib-metadata, Flask
Successfully installed Flask-2.3.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.6 blinker-1.9.0 click-8.1.8 importlib-metadata-8.7.1 itsdangerous-2.2.0 zipp-3.23.0

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
--> Removed intermediate container c0ca4c27f4ed
--> bb71b6086d77
Step 7/15 : FROM python:3.9-slim
--> 085da638e1b8
Step 8/15 : WORKDIR /app
--> Using cache
--> 8b3f3e41932e
Step 9/15 : COPY --from=builder /opt/venv /opt/venv
--> 4fe3e368c0c0
Step 10/15 : ENV PATH="/opt/venv/bin:$PATH"
--> Running in 863b8471fac1
--> Removed intermediate container 863b8471fac1
--> d4b9aca04c26
Step 11/15 : COPY app.py .
--> 60c3124bba0f
Step 12/15 : RUN useradd -m -u 1000 appuser
--> Running in d8aed593bf38
--> Removed intermediate container d8aed593bf38
--> 92372430155e
Step 13/15 : USER appuser
--> Running in 608e6267b515
--> Removed intermediate container 608e6267b515
--> 75c81db3f9fb
Step 14/15 : EXPOSE 5000
--> Running in c328dd1d75e5
--> Removed intermediate container c328dd1d75e5
--> 1779f5599f61
Step 15/15 : CMD ["python", "app.py"]
--> Running in 51e99103d0cd
--> Removed intermediate container 51e99103d0cd
--> f816ffbc171
Successfully built f816ffbc171
Successfully tagged flask-multistage:latest
flask-multistage      latest      f816ffbc171    Less than a second ago    145MB
flask-regular         latest      74b04e3e595c   23 minutes ago           132MB
my-flask-app          1.0        74b04e3e595c   23 minutes ago           132MB
my-flask-app          latest     74b04e3e595c   23 minutes ago           132MB
my-flask-app          v1.0      74b04e3e595c   23 minutes ago           132MB
usernames/my-flask-app 1.0        74b04e3e595c   23 minutes ago           132MB
root@LAPTOP-0B0561KA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# |

```

6.5 Part B — Optimization: Multi-stage build

Multi-stage builds separate build-time dependencies from the runtime image.

Dockerfile.multistage:

```

# STAGE 1: Builder
FROM python:3.9-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /opt/venv && \
    /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime
FROM python:3.9-slim
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY app.py .
# Create non-root user for security
RUN useradd -m -u 1000 appuser
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]

```

Build and compare:

```
- docker build -f Dockerfile.multistage -t flask-multistage:latest .
```

```

root@LAPTOP-988561HA:/mnt/fs/DEVops Lab/Docker Labs/my-flask-app# # Build regular image
docker build -t flask-regular .

# Build multi-stage image
docker build -f Dockerfile.multistage -t flask-multistage .

# Compare sizes
docker images | grep flask-

# Expected output:
# flask-regular      ~250MB
# flask-multistage  ~150MB (40% smaller!)
[+] Building 14.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> transferring context: 277B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabcadale13cf1731b1b
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabcadale13cf1731b1b
=> sha256:fc74438849922d13b0d4b8969a953f842f59ce9d1a0c2c83d710afca286c08 13.88MB / 13.88MB
=> sha256:b3ec39b36ae8c93a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> sha256:38513b07256313b06cd833b0915a633cf4075dc2a07072ab2e3d91090c50 39.70MB / 39.70MB
=> sha256:ea56f685494adf81688322f152d2cfec62115b38dda481c2c450078315be0508 251B / 251B
=> extracting sha256:38513b07256313b06cd833b0915a633cf4075dc2a07072ab2e3d91090c50
=> extracting sha256:b3ec39b36ae8c93a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> extracting sha256:fc74438849922d13b0d4b8969a953f842f59ce9d1a0c2c83d710afca286c08
=> extracting sha256:ea56f685494adf81688322f152d2cfec62115b38dda481c2c450078315be0508
=> [internal] load build context
=> transferring context: 321B
[2/5] WORKDIR /app
[3/5] COPY requirements.txt .
[4/5] RUN pip install --no-cache-dir -r requirements.txt
[5/5] COPY app.py .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:594efeccebaa761ebc9fcee7f77cb7468a58e3d3873c7fab7cc79c52008192f70
=> exporting config sha256:eb7d02d1a01841c2e2a3a0e4a6a0c887f5cda66b08456d43020911df9f8cc0
=> exporting attestation manifest sha256:89f6a3e1197099917f0b08bd42dc360fa29acd083a5b1a4827b853adb86212
=> exporting manifest list sha256:b8d0774f8ca94defe573fe583c272036e5feb5c75c51ab0581045746cfad5628
=> name to docker.io/library/flask-regular:latest
=> unpacking to docker.io/library/flask-regular:latest
[+] Building 9.9s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> transferring dockerfile: 675B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 273B
=> [internal] load build context

```

Figure 40: multistage Dockerfile

```
[*] Building 9.9s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> == transferring dockerfile: 675B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> == transferring context: 273B
=> [internal] load build context
=> == transferring context: 53B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabbacdad13cf1731b1b
=> == resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabbacdad13cf1731b1b
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt
=> [builder 4/5] RUN python -m venv /opt/venv
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv
=> [stage-1 4/5] COPY app.py
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser
=> exporting to image
=> == exporting layers
=> == exporting manifest sha256:0f8423cabe9f190b36897575b09abae63217941dd94fb25a1dd59a147170ffef5
=> == exporting config sha256:acbf80668e9d8f8c2bd7b0f9e3011457f47fb3c34475d17384759b1a4a48690
=> == exporting attestation manifest sha256:5aea4eb451a959efdea07b52aba9e535c40918c340e3ff47d27396e2141c9ded5
=> == exporting manifest list sha256:24462c590c34cfcd8b6273a224a61774b699c8373232a4340866c0b452cdd385d
=> == naming to docker.io/library/flask-multistage:latest
=> == unpacking to docker.io/library/flask-multistage:latest
flask-multistage latest 24462c590c34 2 seconds ago 219MB
flask-regular latest b8d027f48ca0 12 seconds ago 200MB
root@LAPTOP-080561KA:/mnt/ff/DevOps Lab/Docker Labs/my-flask-app# |
```

Images

[Give feedback](#)

Local

My Hub

4.55 GB / 18.96 GB in use

8 Images

Last refresh: 24 minutes ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☒	docker-archestra	latest	14217c1baec5	7 days ago	5.44 GB	
☒	postgres	15	dafc4d8e8369	8 days ago	632.67 MB	
☐	grafana/grafana	latest	9e1e77ade304	8 days ago	994.87 MB	
☐	chromadb/chroma	latest	254a00413c12	12 days ago	804.78 MB	
☒	redis	7	976ab0f0939b	17 days ago	175.28 MB	
☐	prom/prometheus	latest	1f0f50f06aca	1 month ago	502.54 MB	
☐	flask-regular	latest	b8d027f48ca0	13 seconds ago	200.24 MB	
☐	flask-multistage	latest	24462c590c34	3 seconds ago	219.31 MB	

6.6 Part C — Tagging and publishing to Docker Hub

- 1) Login to Docker Hub: `docker login`
- 2) Tag the image for your Docker Hub repository (replace username):
 - `docker tag my-flask-app:latest username/my-flask-app:1.0`
 - `docker tag flask-multistage:latest username/my-flask-app:multistage`

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # 1. Tag the existing local image with your Docker Hub username
docker tag flask-regular:latest durandieltheghost/my-flask-app:1.0

# 2. Push the newly tagged image to the registry
docker push durandieltheghost/my-flask-app:1.0
The push refers to repository [docker.io/durandieltheghost/my-flask-app]
2d7b32a5d2b2: Pushed
b3ec39b36ae8: Pushed
38513bd72563: Pushed
7ab6bf2046ce: Pushed
2d2f025e3be6: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
7cc868acd440: Pushed
7786870935d3: Pushed
1.0: digest: sha256:b8d027f48ca04efe573fe583c272036e5feb5c7c5c51ab0581b45746cfad5628 size: 856
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# |
```

Figure 41: tagging

- 3) Push images:
 - `docker push username/my-flask-app:1.0`
 - `docker push username/my-flask-app:multistage`

```
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # Remove the old local tag now that it's pushed
docker rmi flask-regular:latest

# Clean up all stopped containers
docker container prune
Untagged: flask-regular:latest
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
baec572bca8e85e2788f0a3d5526e967d4cb465b2ee9fda5411592a8c999d68a
dbc1c03149a064a2ad444125e4081579891b7be96f4db22d4a8deef3994f6ba
182c1423f40085d87098668106cbdac6ec167b11fed50e4dbcabc06dfcff3b6f

Total reclaimed space: 876.5kB
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# |
```

GNU nano 7.2

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

6.7 Part D — Node.js App

A minimal Node.js Express app demonstrates a multi-stage pattern for Node.

Project structure:

```
my-node-app/
├─ app.js
├─ package.json
└─ Dockerfile
```

app.js:

```
const express = require('express');
const app = express();
const port = 3000;
app.get('/', (req, res) => res.send('Hello from Node.js Docker!'));
app.get('/health', (req, res) => res.json({ status: 'healthy' }));
app.listen(port, () => console.log(`Server running on port ${port}`));
```

Dockerfile (recommended pattern):

```
# STAGE 1: Builder
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY app.js .
```

```
# STAGE 2: Runtime
```

```

root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app# # 1. Tag the existing local image with your Docker Hub username
docker tag flask-regular:latest durandieltheghost/my-flask-app:1.0

# 2. Push the newly tagged image to the registry
docker push durandieltheghost/my-flask-app:1.0
The push refers to repository [docker.io/durandieltheghost/my-flask-app]
2d7b32a5d2b2: Pushed
b3ec39b36ae8: Pushed
38513bd72563: Pushed
7ab6bf2046ce: Pushed
2d2f025e3be6: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
7cc868acd440: Pushed
7786870935d3: Pushed
1.0: digest: sha256:b8d027f48ca04efe573fe583c272036e5feb5c7c5c51ab0581b45746cfad5628 size: 856
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-flask-app#

```

Figure 42: Files

```

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app .
# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
EXPOSE 3000
CMD ["node", "app.js"]

```

Build and run:

```

- docker build -t my-node-app .
- docker run -d -p 3000:3000 --name node-container my-node-app

```

```

docker build -t my-node-app .

# Run container
docker run -d -p 3000:3000 --name node-container my-node-app

# Test
curl http://localhost:3000
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
f18232174bc9: Verifying Checksum
f18232174bc9: Download complete
f18232174bc9: Pull complete
dd71dde834b5: Download complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
dd71dde834b5: Pull complete
1e5a4c89cee5: Pull complete
25ff2da83641: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
--> ee77c6cd7c18
Step 2/7 : WORKDIR /app
--> Running in 4116bfbe330d
--> Removed intermediate container 4116bfbe330d
--> 2a6f4753b09c
Step 3/7 : COPY package*.json ./
--> b4f1b94e8ff5

```



```

Step 4/7 : RUN npm install --only=production
----> Running in 32fd1a6bd0c0
npm warn config only Use '--omit=dev' to omit dev dependencies from the install.

added 68 packages, and audited 69 packages in 4s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.10.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.10.1
npm notice To update run: npm install -g npm@11.10.1
npm notice
----> Removed intermediate container 32fd1a6bd0c0
----> 986374dbf726
Step 5/7 : COPY app.js .
----> a6478f6875bd
Step 6/7 : EXPOSE 3000
----> Running in 49e2815db54d
----> Removed intermediate container 49e2815db54d
----> 2677ce778f97
Step 7/7 : CMD ["node", "app.js"]
----> Running in 32008eb2070a
----> Removed intermediate container 32008eb2070a
----> 1649f231e201
Successfully built 1649f231e201
Successfully tagged my-node-app:latest
e9d1167327775f38887f748f486422b0dcaec3bbc90fe3e4d7cbae2de42f83ae
curl: (56) Recv failure: Connection reset by peer
root@LAPTOP-0B056IKA:/mnt/f/DevOps Lab/Docker Labs/my-node-app# |

```

6.8 Troubleshooting & Best Practices

- Use `.dockerignore` to keep build context minimal.
 - Prefer pinned dependency versions in `requirements.txt` and `package.json` for reproducibility.
 - Use multi-stage builds to reduce surface area and split build/runtime concerns.
 - Run containers as non-root where possible.
 - Clean up resources: `docker system prune -a` (careful: removes images/containers not in use).
-

6.9 Conclusion

This lab demonstrates a full container lifecycle: local app creation, Dockerfile authoring, optimization with `.dockerignore` and multi-stage builds, image inspection, tagging, and publishing to Docker Hub.

6.10 Additional Resources

- [Dockerfile Reference](#)
- [Best Practices for Writing Dockerfiles](#)
- [Multi-stage Build Documentation](#)
- [Docker Hub Official Repositories](#)

7 Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

7.1 Table of Contents

1. [Part 1: Docker Volumes](#)
 2. [Part 2: Environment Variables](#)
 3. [Part 3: Docker Monitoring](#)
 4. [Part 4: Docker Networks](#)
 5. [Part 5: Complete Real-World Example](#)
 6. [Quick Reference Cheatsheet](#)
 7. [Practice Exercises](#)
 8. [Conclusion](#)
 9. [Additional Resources](#)
-

7.2 Part 1: Docker Volumes - Persistent Data Storage

7.2.1 Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data
docker run -it --name test-container ubuntu /bin/bash

# Inside container:
echo "Hello World" > /data/message.txt
cat /data/message.txt # Shows "Hello World"
exit

# Restart container
docker start test-container
docker exec test-container cat /data/message.txt
# ERROR: File doesn't exist! Data was lost.
```

Solution: Docker Volumes

```
PS F:\DevOps Theory Lab\Docker Labs> docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
root@d6903bcae98f:/#
```

```

PS F:\DevOps Theory Lab\Docker Labs> docker run -it --name test-container ubuntu /bin/bash
root@e729f3cf3014:/# # Inside the container:
mkdir /data
echo "Hello World" > /data/message.txt
exit
exit
PS F:\DevOps Theory Lab\Docker Labs> docker rm test-container
test-container
PS F:\DevOps Theory Lab\Docker Labs> docker run -it --name test-container ubuntu /bin/bash
>> cat /data/message.txt
>> # Result: cat: /data/message.txt: No such file or directory
root@d3187f84e2ad:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@d3187f84e2ad:/#

```

7.2.2 Lab 2: Volume Types

7.2.2.1 1. Anonymous Volumes

Create anonymous volume (auto-generated name)

```
docker run -d -v /app/data --name web1 nginx
```

Check volume

```
docker volume ls
```

Shows: anonymous volume with random hash

Inspect container to see volume mount

```
docker inspect web1 | grep -A 5 Mounts
```

```

PS F:\DevOps Theory Lab\Docker Labs> docker volume create mydata
mydata
PS F:\DevOps Theory Lab\Docker Labs> docker run -d -v mydata:/app/data --name web2 nginx
b95dd3ada860f347c38ff24fc2cc073985499f354fcb0826e84af63b82d55cfe
PS F:\DevOps Theory Lab\Docker Labs> docker volume ls
>> docker volume inspect mydata
DRIVER      VOLUME NAME
local       217a907ddbe9369479b8f491c038613427cda29ea393db34d65f4f329b4d2910
local       1572096f19a3a2921bb91b6f081eb6789f18fd3e80e28f286d53a120e7a96f7e
local       advanced_pgdata
local       cloudscape_project_aws_finance_data
local       cloudscape_project_aws_production_data
local       cloudscape_project_aws_shared_data
local       cloudscape_project_azure_data
local       cloudscape_project_neo4j_data
local       cloudscape_project_neo4j_logs
local       mydata
[
  {
    "CreatedAt": "2026-03-20T03:28:15Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
PS F:\DevOps Theory Lab\Docker Labs>

```

Figure 43: Anonymous Volume

7.2.2.2 2. Named Volumes

Create named volume

```
docker volume create mydata
```

Use named volume

```
docker run -d -v mydata:/app/data --name web2 nginx
```

List volumes

```
docker volume ls
```

Shows: mydata

Inspect volume

```
docker volume inspect mydata
```

```
PS F:\DevOps Theory Lab\Docker Labs> mkdir myapp-data
```

```
Directory: F:\DevOps Theory Lab\Docker Labs
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	20-03-2026 08:59 AM		myapp-data

```
PS F:\DevOps Theory Lab\Docker Labs> docker run -d -v "${PWD}/myapp-data:/app/data" --name web3 nginx
927e55f38d625c1c822be857b7a7f5bf2b9d1a1fa24f1f5bcbf19db83f25af8b
```

```
PS F:\DevOps Theory Lab\Docker Labs> echo "Hello from Windows" > ./myapp-data/host-file.txt
```

```
PS F:\DevOps Theory Lab\Docker Labs> docker exec web3 cat /app/data/host-file.txt
```

```
♦♦Hello from Windows
```

```
PS F:\DevOps Theory Lab\Docker Labs> █
```

```

PS F:\DevOps Theory Lab\Docker Labs> docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
e122248aa49c: Pull complete
9575b8625e4a: Pull complete
cf107c1a2b33: Pull complete
177fa0193b5c: Pull complete
a1297c1feceb: Pull complete
0fcc630d3abc: Pull complete
f57a6efc32c6: Pull complete
f3cd727943cf: Pull complete
183dada87886: Pull complete
df11b1bcbaee: Pull complete
ca475193795e: Pull complete
096d7465fe0a: Download complete
3330278a99c6: Download complete
Digest: sha256:0f34c70018dcbde655ba3eaba3d33c02198d392b9364974462eeee13a903af385
Status: Downloaded newer image for mysql:8.0
250a1e53c99d22e90ddfad77fa33ac3ba24d18ae5f92fd10d313a7bfb5894b9
PS F:\DevOps Theory Lab\Docker Labs> docker volume ls
DRIVER      VOLUME NAME
local       1572096f19a3a2921bb91b6f081eb6789f18fd3e80e28f286d53a120e7a96f7e
local       advanced_pgdata
local       cloudscape_project_aws_finance_data
local       cloudscape_project_aws_production_data
local       cloudscape_project_aws_shared_data
local       cloudscape_project_azure_data
local       cloudscape_project_neo4j_data
local       cloudscape_project_neo4j_logs
local       mydata
local       mysql-data
PS F:\DevOps Theory Lab\Docker Labs> docker stop mysql-db
>> docker rm mysql-db
mysql-db
mysql-db
PS F:\DevOps Theory Lab\Docker Labs>

```

7.2.2.3 3. Bind Mounts (Host Directory)

```

# Create directory on host
mkdir ~/myapp-data

# Mount host directory to container
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host
echo "From Host" > ~/myapp-data/host-file.txt

# Check in container
docker exec web3 cat /app/data/host-file.txt
# Shows: From Host

```

7.2.3 Lab 3: Practical Volume Examples

7.2.3.1 Example 1: Database with Persistent Storage

```

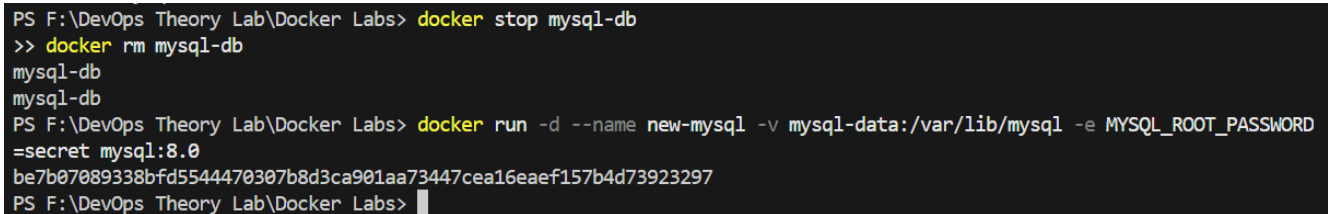
# MySQL with named volume
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \

```

```
mysql:8.0

# Check data persists
docker stop mysql-db
docker rm mysql-db

# New container with same volume
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
# Data is preserved!
```



```
PS F:\DevOps Theory Lab\Docker Labs> docker stop mysql-db
>> docker rm mysql-db
mysql-db
mysql-db
PS F:\DevOps Theory Lab\Docker Labs> docker run -d --name new-mysql -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:8.0
be7b07089338bfd5544470307b8d3ca901aa73447cea16eae157b4d73923297
PS F:\DevOps Theory Lab\Docker Labs>
```

Figure 44: MySQL Volume Persistence

7.2.3.2 Example 2: Web App with Configuration Files

```
# Create config directory
mkdir ~/nginx-config

# Create nginx config file
echo 'server {
    listen 80;
    server_name localhost;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx

# Test
curl http://localhost:8080
```

7.2.4 Lab 4: Volume Management Commands

```
# List all volumes
docker volume ls

# Create a volume
docker volume create app-volume
```

```

PS F:\DevOps Theory Lab\Docker Labs> mkdir nginx-config
>> cd nginx-config

Directory: F:\DevOps Theory Lab\Docker Labs


Mode                LastWriteTime         Length Name
----                -
d-----          20-03-2026   09:13 AM             nginx-config

PS F:\DevOps Theory Lab\Docker Labs\nginx-config> $config = 'server {
>>     listen 80;
>>     server_name localhost;
>>     location / {
>>         return 200 "Hello from mounted config!";
>>     }
>> }'
>> $config | Out-File -FilePath nginx.conf -Encoding ascii
PS F:\DevOps Theory Lab\Docker Labs\nginx-config> cd ..
>> docker run -d --name nginx-custom -p 8080:80 -v "${PWD}/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf" nginx
56f996fb4c8e0995a72f5fb4027f47126c1f1curl http://localhost:80809
>> # Expected Output: Hello from mounted config!

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): n

```

Figure 45: Nginx Custom Config

```

# Inspect volume details
docker volume inspect app-volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume

```



```

>> docker volume ls
>>
>> # Create a new volume specifically for this lab
>> docker volume create lab4-vol
DRIVER      VOLUME NAME
local       1572096f19a3a2921bb91b6f081eb6789f18fd3e80e28f286d53a120e7a96f7e
local       advanced_pgdata
local       app-volume
local       cloudscape_project_aws_finance_data
local       cloudscape_project_aws_production_data
local       cloudscape_project_aws_shared_data
local       cloudscape_project_azure_data
local       cloudscape_project_neo4j_data
local       cloudscape_project_neo4j_logs
local       mydata
local       mysql-data
lab4-vol
PS F:\DevOps Theory Lab\ Docker Labs> docker volume inspect lab4-vol
[
  {
    "CreatedAt": "2026-03-20T04:01:53Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/lab4-vol/_data",
    "Name": "lab4-vol",
    "Options": null,
    "Scope": "local"
  }
]
PS F:\DevOps Theory Lab\ Docker Labs> docker run -d --name lab4-container -v lab4-vol:/app/data nginx
914534c44f4a670da8847b2fb96963df8e3f1626a9e5b87a4aaeeba5f2b5ad28
PS F:\DevOps Theory Lab\ Docker Labs> echo "This file was created on Windows and sent to Docker" > test-lab4.txt
PS F:\DevOps Theory Lab\ Docker Labs> docker cp test-lab4.txt lab4-container:/app/data/
Successfully copied 2.05kB to lab4-container:/app/data/
PS F:\DevOps Theory Lab\ Docker Labs> docker cp test-lab4.txt lab4-container:/app/data/
Successfully copied 2.05kB to lab4-container:/app/data/
PS F:\DevOps Theory Lab\ Docker Labs>
PS F:\DevOps Theory Lab\ Docker Labs> docker exec lab4-container ls /app/data/
>> docker exec lab4-container cat /app/data/test-lab4.txt
test-lab4.txt
💎This file was created on Windows and sent to Docker
PS F:\DevOps Theory Lab\ Docker Labs>

```

7.3 Part 2: Environment Variables

7.3.1 Lab 1: Setting Environment Variables

Method 1: Using -e flag

Single variable

```

docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app

```

```

PS F:\DevOps Theory Lab\ Docker Labs> docker run -d --name app-env-test -e DB_HOST=10.0.0.5 -e API_KEY=abc12345 -e DEBUG=true nginx
539385fdb52a9a4a9a8780c44914d406ba86df0459c8524db5ebacc7049a2520

```

Figure 46: Env Var -e Flag

Method 2: Using --env-file

```
# Create .env file
echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
```

```
# Use env file
docker run -d \
  --env-file .env \
  --name app2 \
  my-app
```

```
PS F:\DevOps Theory Lab\ Docker Labs> "DB_PORT=5432`nDB_USER=admin`nLOG_LEVEL=info" | Out-File -FilePath .env -Encoding ascii
PS F:\DevOps Theory Lab\ Docker Labs> docker run -d --name app-env-file --env-file .env nginx
19f2c5e3c0ceefa0908d8d0bdca52c4468255d55451791d9c580cb1de341546c
PS F:\DevOps Theory Lab\ Docker Labs> █
```

Figure 47: Env File Integration

7.3.2 Lab 2: Environment Variables in Applications

Python Flask Example

```
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port)
```

Dockerfile with Environment Variables

```
FROM python:3.9-slim
ENV PORT=5000
ENV DEBUG=false
EXPOSE 5000
CMD ["python", "app.py"]
```

```

PS F:\DevOps Theory Lab> cd DockerLabs\lab-5
PS F:\DevOps Theory Lab\docker> docker build -t flask-env-app .
[+] Building 14.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 426B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacda1e13cf1731b1b
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacda1e13cf1731b1b
=> sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 251B / 251B
=> sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d
=> extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 669B
=> [2/4] WORKDIR /app
=> [3/4] RUN pip install flask
=> [4/4] COPY app.py .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:964a181762f1c0f296962d57c73a0c7e30121e0c8eee1f7ede12de902739921d
=> exporting config sha256:684f5e42b59707066c881fa8ca3e801d8a7553b3c4368e8d74bf54fda1630ndc
=> exporting attestation manifest sha256:2ebf013d64a3529ed428086e763715ddb807062338f5c5e6a6341c60c7951bdc
=> exporting manifest list sha256:61b02984abf47d97d3afe039215bbcd0f84fa2fb95c04c0b1f947f7e055db355
=> naming to docker.io/library/flask-env-app:latest
=> unpacking to docker.io/library/flask-env-app:latest
0.1s
0.0s
4.3s
0.0s
0.0s
0.0s
5.0s
0.0s
0.8s
1.9s
1.8s
3.4s
0.8s
0.1s
0.5s
0.0s
0.0s
0.2s
4.3s
0.0s
0.5s
0.3s
0.0s
0.0s
0.0s
0.0s
0.0s
0.0s
0.0s
0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lb72alpm6i3gxix1pomm31edf
PS F:\DevOps Theory Lab\docker>

```

```

PS F:\DevOps Theory Lab\docker> docker run -d `
>> --name flask-app `
>> -p 5000:8080 `
>> -e DATABASE_HOST="prod-db.example.com" `
>> -e DEBUG="true" `
>> -e PORT="8080" `
>> flask-env-app
00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af
PS F:\DevOps Theory Lab\docker> docker exec flask-app env
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=00d10b6631a8
DATABASE_HOST=prod-db.example.com
DEBUG=true
PORT=8080
LANG=C.UTF-8
GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568
PYTHON_VERSION=3.9.25
PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834
PYTHONUNBUFFERED=1
PYTHONDONTWRITEBYTECODE=1
HOME=/root
PS F:\DevOps Theory Lab\docker> docker exec flask-app printenv DATABASE_HOST
prod-db.example.com

```

7.4 Part 3: Docker Monitoring

7.4.1 Lab 1: Basic Monitoring Commands

Live stats for all containers

```
docker stats
```

JSON output

```
docker stats --format json --no-stream
```

```

CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
00d10b6631a8   flask-app                          0.08%    44.11MiB / 7.761GiB 0.56%    1.82kB / 859B   0B / 0B     3
15fcfd3042eb   advanced_web_portal               0.00%    7.25MiB / 256MiB    2.83%    2.21kB / 126B   0B / 0B     5
926598a65955   advanced_fastapi                  0.68%    292.5MiB / 512MiB   57.13%    2.05kB / 126B   0B / 0B     6
56b62d504d72   advanced_redis                    0.37%    8.391MiB / 7.761GiB 0.11%    2.01kB / 126B   0B / 0B     6
2dfdd973b84b   advanced_postgres                 0.00%    31.01MiB / 512MiB   6.06%    2.14kB / 126B   0B / 0B     6
a0cd6439907f   cloudscape_localstack_aws        0.04%    337.7MiB / 7.761GiB 4.25%    358kB / 310kB   0B / 0B     9
cc68ad2b7822   cloudscape_redis_cache            0.14%    11.52MiB / 7.761GiB 0.14%    2.74kB / 126B   0B / 0B     6
37a60bd3a682   cloudscape_neo4j_engine           1.21%    2.058GiB / 7.761GiB 26.52%    18.9MB / 243kB   0B / 0B    98
e73f838d6ecc   cloudscape_azurite_azure          0.00%    59.5MiB / 7.761GiB 0.75%    18.7kB / 14kB    0B / 0B    11

PS F:\DevOps Theory Lab\DockeLab\lab-5> docker stats --no-stream
CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
00d10b6631a8   flask-app                          0.11%    44.11MiB / 7.761GiB 0.56%    1.82kB / 859B   0B / 0B     3
15fcfd3042eb   advanced_web_portal               0.00%    7.25MiB / 256MiB    2.83%    2.21kB / 126B   0B / 0B     5
926598a65955   advanced_fastapi                  0.68%    292.5MiB / 512MiB   57.13%    2.05kB / 126B   0B / 0B     6
56b62d504d72   advanced_redis                    0.56%    8.395MiB / 7.761GiB 0.11%    2.01kB / 126B   0B / 0B     6
2dfdd973b84b   advanced_postgres                 0.00%    30.89MiB / 512MiB   6.03%    2.14kB / 126B   0B / 0B     6
a0cd6439907f   cloudscape_localstack_aws        0.04%    337.7MiB / 7.761GiB 4.25%    358kB / 310kB   0B / 0B     9
cc68ad2b7822   cloudscape_redis_cache            0.15%    11.51MiB / 7.761GiB 0.14%    2.74kB / 126B   0B / 0B     6
37a60bd3a682   cloudscape_neo4j_engine           1.39%    2.058GiB / 7.761GiB 26.52%    18.9MB / 243kB   0B / 0B    98
e73f838d6ecc   cloudscape_azurite_azure          0.00%    59.5MiB / 7.761GiB 0.75%    18.7kB / 14kB    0B / 0B    11
PS F:\DevOps Theory Lab\DockeLab\lab-5>

```

7.4.2 Lab 2: docker top - Process Monitoring

```

# View processes in container
docker top container-name

```

```

PS F:\DevOps Theory Lab\DockeLab\lab-5> docker top flask-app
UID          PID          PPID         C           STIME        TTY          TIME         CMD
root         79968        79945        0           13:37        ?           00:00:00     python app.py
root         80008        79968        0           13:37        ?           00:00:18     /usr/local/bin/python /app/app.py
PS F:\DevOps Theory Lab\DockeLab\lab-5> docker top flask-app -ef
UID          PID          PPID         C           STIME        TTY          TIME         CMD
root         79968        79945        0           13:37        ?           00:00:00     python app.py
root         80008        79968        0           13:37        ?           00:00:18     /usr/local/bin/python /app/app.py
PS F:\DevOps Theory Lab\DockeLab\lab-5>

```

Figure 48: Docker Top

7.4.3 Lab 3: docker logs - Application Logs

```

# Follow logs (like tail -f)
docker logs -f container-name

```

7.4.4 Lab 4: Container Inspection

```

# Detailed container info
docker inspect container-name

```

```

PS F:\DevOps Theory Lab\DockerLabs\lab-5> docker logs flask-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.17.0.2:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 103-259-011
172.17.0.1 - - [03/Apr/2026 13:35:50] "GET /config HTTP/1.1" 200 -
PS F:\DevOps Theory Lab\DockerLabs\lab-5> docker logs -f flask-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.17.0.2:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 103-259-011
172.17.0.1 - - [03/Apr/2026 13:35:50] "GET /config HTTP/1.1" 200 -
PS F:\DevOps Theory Lab\DockerLabs\lab-5> docker logs --tail 20 -t flask-app
2026-04-03T13:35:12.226855918Z * Serving Flask app 'app'
2026-04-03T13:35:12.226893345Z * Debug mode: on
2026-04-03T13:35:12.262453226Z WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-04-03T13:35:12.262503098Z * Running on all addresses (0.0.0.0)
2026-04-03T13:35:12.262512976Z * Running on http://127.0.0.1:8080
2026-04-03T13:35:12.262520711Z * Running on http://172.17.0.2:8080
2026-04-03T13:35:12.262529143Z Press CTRL+C to quit
2026-04-03T13:35:12.262812652Z * Restarting with stat
2026-04-03T13:35:12.761159356Z * Debugger is active!
2026-04-03T13:35:12.761759892Z * Debugger PIN: 103-259-011
2026-04-03T13:35:50.554777857Z 172.17.0.1 - - [03/Apr/2026 13:35:50] "GET /config HTTP/1.1" 200 -
PS F:\DevOps Theory Lab\DockerLabs\lab-5>

```

Figure 49: Docker Logs

```

    ],
    "Image": "flask-env-app",
    "Volumes": null,
    "WorkingDir": "/app",
    "Entrypoint": null,
    "Labels": {},
    "StopTimeout": 1
  },
  "NetworkSettings": {
    "SandboxID": "7a6e781eb1fdb6c9c35d557545a71bf599895a26534f7cd4c5a365a9a3727846",
    "SandboxKey": "/var/run/docker/netns/7a6e781eb1fd",
    "Ports": {
      "8080/tcp": [
        {
          "HostIp": "0.0.0.0",
          "HostPort": "5000"
        },
        {
          "HostIp": "::",
          "HostPort": "5000"
        }
      ]
    }
  },
  "Networks": {
    "bridge": {
      "IPAMConfig": null,
      "Links": null,
      "Aliases": null,
      "DriverOpts": null,
      "GwPriority": 0,
      "NetworkID": "ff95cfcf4365025f71dc8cc60c64103042087197ed82b146240ea57b70e7471c",
      "EndpointID": "1125a19c5733e7b4519075bad5fc63fbc815517140c82a77a69096e41547776f",
      "Gateway": "172.17.0.1",
      "IPAddress": "172.17.0.2",
      "MacAddress": "5a:a7:f5:fd:da:1e",
      "IPPrefixLen": 16,
      "IPv6Gateway": "",
      "GlobalIPv6Address": "",
      "GlobalIPv6PrefixLen": 0,
      "DNSNames": null
    }
  }
}

```

```
PS F:\DevOps Theory Lab\docker\labs\lab-5> docker inspect flask-app
```

```
[
  {
    "Id": "00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af",
    "Created": "2026-04-03T13:35:11.130619222Z",
    "Path": "python",
    "Args": [
      "app.py"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 79968,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-03T13:35:11.295638407Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:61b92984abf47d97d3afe039215bbcd0f84fa2fb95c04c8b1f947f7e055db355",
    "ResolvConfPath": "/var/lib/docker/containers/00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af/hostname",
    "HostsPath": "/var/lib/docker/containers/00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af/hosts",
    "LogPath": "/var/lib/docker/containers/00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af/00d10b6631a809875d5c4df20ee3fe8847eb8381b5eb54c713a4f3a867e618af-json.log",
    "Name": "/flask-app",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",
      "PortBindings": {
        "8080/tcp": [
          {
            "HostIp": "",
            "HostPort": "5000"
          }
        ]
      }
    },
    "RestartPolicy": {
      "Name": "no",
      "MaximumRetryCount": 0
    }
  }
]
```

```

"AutoRemove": false,
"VolumeDriver": "",
"VolumesFrom": null,
"ConsoleSize": [
    45,
    183
],
"CapAdd": null,
"CapDrop": null,
"CgroupnsMode": "host",
"Dns": null,
"DnsOptions": [],
"DnsSearch": [],
"ExtraHosts": null,
"GroupAdd": null,
"IpcMode": "private",
"Cgroup": "",
"Links": null,
"OomScoreAdj": 0,
"PidMode": "",
"Privileged": false,
"PublishAllPorts": false,
"ReadonlyRootfs": false,
"SecurityOpt": null,
"UTSMode": "",
"UsernsMode": "",
"ShmSize": 67108864,
"Runtime": "runc",
"Isolation": "",
"CpuShares": 0,
"Memory": 0,
"NanoCpus": 0,
"CgroupParent": "",
"BlkioWeight": 0,
"BlkioWeightDevice": [],
"BlkioDeviceReadBps": [],
"BlkioDeviceWriteBps": [],
"BlkioDeviceReadIOps": [],
"BlkioDeviceWriteIOps": [],
"CpuPeriod": 0,
"CpuQuota": 0,
"CpuRealtimePeriod": 0,
"CpuRealtimeRuntime": 0,
"CpusetCpus": "",
"CpusetMems": "",
"Devices": [],
"DeviceCgroupRules": null,
"DeviceRequests": null,
"MemoryReservation": 0,
"MemorySwap": 0,
"MemorySwappiness": null,
"OomKillDisable": false,
"PidsLimit": null,
"Ulimits": [],

```



```
PS F:\DevOps Theory Lab\DockerLabs\lab-5> docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' flask-app
172.17.0.2
PS F:\DevOps Theory Lab\DockerLabs\lab-5> docker inspect --format='{{.State.Status}}' flask-app
running
PS F:\DevOps Theory Lab\DockerLabs\lab-5>
```

7.4.5 Lab 5: Events Monitoring

```
# Monitor Docker events in real-time
```

docker events

[illegible]


```

PS F:\DevOps Theory Lab\DockerLabs\lab-5> .\monitor.ps1
=== Docker Monitoring Dashboard ===
Time: 04/03/2026 23:47:03

1. Running Containers:
NAMES                STATUS                PORTS
flask-app             Up 5 hours            0.0.0.0:5000->8080/tcp, [::]:5000->8080/tcp
advanced_web_portal   Up 7 hours            0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
advanced_fastapi       Up 7 hours (healthy)  0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp
advanced_redis         Up 7 hours            0.0.0.0:6380->6379/tcp, [::]:6380->6379/tcp
advanced_postgres      Up 7 hours (healthy)  5432/tcp
cloudscape_localstack_aws Up 7 hours (healthy)  0.0.0.0:4510-4559->4510-4559/tcp, [::]:4510-4559->4510-4559/tcp, 0.0.0.0:4566->4566/tcp, [::]:4566->4566/t
cp
cloudscape_redis_cache Up 7 hours (healthy)  0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
cloudscape_neo4j_engine Up 7 hours (healthy)  0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp
cloudscape_azurite_azure Up 7 hours (unhealthy) 0.0.0.0:10000-10002->10000-10002/tcp, [::]:10000-10002->10000-10002/tcp

2. Resource Usage:
NAME                CPU %    MEM USAGE / LIMIT
flask-app           0.09%    44.11MiB / 7.761GiB
advanced_web_portal 0.00%    7.25MiB / 256MiB
advanced_fastapi     0.68%    292.5MiB / 512MiB
advanced_redis       0.39%    8.391MiB / 7.761GiB
advanced_postgres    0.00%    30.89MiB / 512MiB
cloudscape_localstack_aws 0.03%    337.7MiB / 7.761GiB
cloudscape_redis_cache 4.68%    11.65MiB / 7.761GiB
cloudscape_neo4j_engine 1.36%    2.072GiB / 7.761GiB
cloudscape_azurite_azure 0.00%    59.51MiB / 7.761GiB

3. System Info:
TYPE    TOTAL    ACTIVE    SIZE    RECLAIMABLE
Images   19       10      13.18GB  6.213GB (47%)
Containers 10       9       462.6MB  81.92kB (0%)
Local Volumes 16       2       799.6MB  751.7MB (94%)
Build Cache 91       0       2.116GB  878.8MB

PS F:\DevOps Theory Lab\DockerLabs\lab-5>

```

7.5 Part 4: Docker Networks

7.5.1 Lab 1: Understanding Docker Network Types

List Networks

```
docker network ls
```

7.5.2 Lab 2: Network Types Explained

1. **Bridge (Default):** Isolated internal network.
2. **Host:** Shares host network stack.
3. **None:** No network interfaces.
4. **Overlay:** Multi-host communication (Swarm).

7.6 Part 5: Complete Real-World Example

1. Create network

```
docker network create myapp-network
```

2. Start database with volume

```
docker run -d --name postgres --network myapp-network -e POSTGRES_PASSWORD=mysecretpassword -v
↳ postgres-data:/var/lib/postgresql/data postgres:15
```

3. Start Flask app

```
docker run -d --name flask-app --network myapp-network -p 5000:5000 -v $(pwd)/app:/app -e
↳ DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" flask-app:latest
```

7.7 Quick Reference Cheatsheet

Category	Commands
Volumes	<code>docker volume create</code> , <code>docker run -v</code> , <code>docker volume ls</code>
Env Vars	<code>docker run -e</code> , <code>docker run --env-file</code>
Monitoring	<code>docker stats</code> , <code>docker logs -f</code> , <code>docker top</code> , <code>docker events</code>
Networks	<code>docker network create</code> , <code>docker network connect</code>

7.8 Practice Exercises

1. **Database Backup:** Practice copying data from a volume using `docker cp`.
 2. **Multi-Service Setup:** Create a web app and DB on a custom network.
 3. **Log Analysis:** Use `docker logs -f` to filter output.
-

7.9 Key Takeaways

- **Volumes** persist data beyond container lifecycle.
 - **Environment Variables** configure containers dynamically.
 - **Monitoring** helps debug and optimize resource usage.
 - **Networks** enable secure container communication.
-

7.10 Additional Resources

- [Manage data in Docker](#)
- [Docker Networking Overview](#)
- [Docker Statistics Reference](#)

8 Experiment 6: Docker Run vs. Docker Compose

8.1 Table of Contents

1. [PART A – THEORY](#)
 2. [PART B – PRACTICAL TASK](#)
 3. [PART C – CONVERSION & BUILD-BASED TASKS](#)
 4. [PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE](#)
 5. [MULTI-CONTAINER APPLICATION INVOLVING DOCKER SWARM](#)
 6. [Conclusion](#)
 7. [Additional Resources](#)
-

8.2 PART A – THEORY

8.2.1 1. Objective

To understand the relationship between `docker run` and **Docker Compose**, and to compare their configuration syntax and use cases.

8.2.2 2. Background Theory

8.2.2.1 2.1 Docker Run (Imperative Approach) The `docker run` command is used to create and start a container from an image. It requires explicit flags for configuration. This approach is **imperative**, meaning you provide step-by-step instructions.

Example:

```
docker run -d --name my-nginx -p 8080:80 nginx:alpine
```

8.2.2.2 2.2 Docker Compose (Declarative Approach) Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes. Instead of multiple `docker run` commands, a single command is used:

```
docker compose up -d
```

Compose is **declarative**, meaning you define the desired state of the application.

8.2.3 3. Comparison: Docker Run vs. Docker Compose

Feature	Docker Run	Docker Compose
Approach	Imperative (Step-by-step)	Declarative (Desired state)
Configuration	Command-line flags	YAML file
Complexity	Simple for single containers	Efficient for multi-service apps
Management	Manual tracking	Version-controlled configuration
Service Links	Manual network links	Automatic internal DNS

8.3 PART B – PRACTICAL TASK

8.3.1 Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

```
docker run -d --name lab-nginx -p 8081:80 nginx:alpine
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml`:

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> # Run Nginx
>> docker run -d --name lab-nginx -p 8081:80 -v ${PWD}/html:/usr/share/nginx/html nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
612c0c1df4c5: Pull complete
82736a35d0e7: Pull complete
aee4e54b3865: Pull complete
583599bb7d38: Pull complete
6a0ac1617861: Pull complete
453da7dbc73e: Pull complete
4a8b0b2a5b19: Pull complete
781ff50d2644: Pull complete
6192e1e6a438: Download complete
e8fc446e336c: Download complete
Digest: sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
Status: Downloaded newer image for nginx:alpine
8a21caaed962d6ae0d1b1cc503e9add9c0f3e4bdd16db096999cb51026361f38
PS F:\DevOps Theory Lab\DockerLabs\lab-6>

```

Figure 50: Docker Run Nginx

```

version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"

```

8.4 PART C – CONVERSION & BUILD-BASED TASKS

8.4.1 Task 2: Multi-Container Application (WordPress + MySQL)

8.4.1.1 Using Docker Compose (Structured way)

```

version: '3.8'
services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

```

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> # --- Part 2: Docker Compose ---
>> mkdir Task1; cd Task1; cp -r ../html .
>> # Correct PowerShell syntax for creating the file
>> @'
>> version: '3.8'
>> services:
>>   nginx:
>>     image: nginx:alpine
>>     container_name: lab-nginx
>>     ports:
>>       - "8081:80"
>>     volumes:
>>       - ../html:/usr/share/nginx/html
>> '@ | Set-Content -Path docker-compose.yml

Directory: F:\DevOps Theory Lab\DockerLabs\lab-6

Mode                LastWriteTime         Length Name
----                -
d-----          01-05-2026  11:06 PM             Task1

PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task1> docker compose up -d
>> docker compose ps
time="2026-05-01T23:06:58+05:30" level=warning msg="F:\DevOps Theory Lab\DockerLabs\lab-6\Task1\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✓ Network task1_default Created 0.1s
✓ Container lab-nginx Created 0.1s
time="2026-05-01T23:06:59+05:30" level=warning msg="F:\DevOps Theory Lab\DockerLabs\lab-6\Task1\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME        IMAGE        COMMAND                  SERVICE    CREATED          STATUS          PORTS
lab-nginx   nginx:alpine "/docker-entrypoint...." nginx      Less than a second ago Up Less than a second 0.0.0.0:8081->80/tcp,
[::]:8081->80/tcp
PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task1>

```

Figure 51: Docker Compose Nginx

```
volumes:
  mysql_data:
```

```
PS F:\DevOps Theory Lab\dockerLabs\lab-6> # --- Part 2: Docker Compose ---
>> mkdir Task2; cd Task2
>> @'
>> version: '3.8'
>> services:
>>   mysql:
>>     image: mysql:5.7
>>     environment:
>>       MYSQL_ROOT_PASSWORD: secret
>>       MYSQL_DATABASE: wordpress
>>     volumes:
>>       - mysql_data:/var/lib/mysql
>>   wordpress:
>>     image: wordpress:latest
>>     ports:
>>       - "8082:80"
>>     environment:
>>       WORDPRESS_DB_HOST: mysql
>>       WORDPRESS_DB_PASSWORD: secret
>>     depends_on:
>>       - mysql
>> volumes:
>>   mysql_data:
>> '@ | Set-Content -Path docker-compose.yml
```

Directory: F:\DevOps Theory Lab\dockerLabs\lab-6

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	01-05-2026 11:10 PM		Task2

```

PS F:\DevOps Theory Lab\dockerLabs\lab-6> # --- Part 1: Docker Run ---
>> docker network create wp-net
>> docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=wordpress mysql:5.7
>> docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret wordpress:latest
cbc4127a41969c273322f93afa731295ba2b6eb74af4ef80dfbae3fa315f562
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
6b95a940e7b6: Pull complete
1c56c3d4ce74: Pull complete
20e4dcae4c69: Pull complete
e9f03a1c24ce: Pull complete
ae71319cb779: Pull complete
68c3898c2015: Pull complete
ffc89e9df888: Pull complete
064b2d298fba: Pull complete
df9a4d85569b: Pull complete
90986bb8de6e: Pull complete
43d05e938198: Pull complete
9eb6a52b8441: Download complete
96889c47a44a: Download complete
Digest: sha256:4bc6bc963e6d844345367cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
9473d3588b719640ac4248392ec77fd8433977ae063242c9d341124ac73ddf68
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
c1343810066f: Pull complete
79b094a6a742: Pull complete
2a0b801d7f1e: Pull complete
a923b3cdaa21: Pull complete
d0d4ea2c39f4: Pull complete
4f4fb700ef54: Pull complete
f7417fc69aeb: Pull complete
d720ae4e5c6c: Pull complete
a5f587e05427: Pull complete
fdcfa2b8fc35: Pull complete
cac4a54c06df: Pull complete

PS F:\DevOps Theory Lab\dockerLabs\lab-6\Task2> docker compose up -d
time="2026-05-01T23:10:54+05:30" level=warning msg="F:\DevOps Theory Lab\dockerLabs\lab-6\Task2\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 4/4
✓ Network task2_default Created 0.1s
✓ Volume task2_mysql_data Created 0.0s
✓ Container task2-mysql-1 Created 0.1s
✓ Container task2-wordpress-1 Created 0.2s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint task2-wordpress-1 (5599f3ae516ebfe400c61d9b2b702cbae292859cc6b96f9c4576d03caffddaa): Bind for 0.0.0.0:8082 failed: port is already allocated
PS F:\DevOps Theory Lab\dockerLabs\lab-6\Task2>

```

8.5 PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE

8.5.1 Task 3: Replace Standard Image with Dockerfile

Step 1: Create Dockerfile

```

FROM node:18-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]

```

Step 2: Create docker-compose.yml

```

version: '3.8'
services:
  nodeapp:
    build: .

```

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> # Create directory and enter it
>> mkdir Task3_P1; cd Task3_P1
>> # Create the equivalent docker-compose.yml
>> @'
>> version: '3.8'
>> services:
>>   webapp:
>>     image: node:18-alpine
>>     container_name: webapp
>>     ports:
>>       - "5000:5000"
>>     environment:
>>       - APP_ENV=production
>>       - DEBUG=false
>>     restart: unless-stopped
>> '@ | Set-Content -Path docker-compose.yml

Directory: F:\DevOps Theory Lab\DockerLabs\lab-6

Mode                LastWriteTime         Length Name
----                -
d-----          01-05-2026  11:14 PM             Task3_P1

PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task3_P1> docker compose up -d
time="2026-05-01T23:15:33+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Task3_P1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 8/8
✓ Image node:18-alpine      Pulled                                10.7s
✓ Network task3_p1_default  Created                                0.0s
✓ Container webapp          Created                                0.2s
PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task3_P1>

```

Figure 52: Dockerfile Creation

```

container_name: custom-node-app
ports:
  - "3000:3000"

```



```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> # Create directory and enter it
>> mkdir Task3_P2; cd Task3_P2
>>
>> # Create the multi-service docker-compose.yml
>> @'
>> version: '3.8'
>> services:
>>   postgres-db:
>>     image: postgres:15
>>     container_name: postgres-db
>>     networks:
>>       - app-net
>>     environment:
>>       - POSTGRES_USER=admin
>>       - POSTGRES_PASSWORD=secret
>>     volumes:
>>       - pgdata:/var/lib/postgresql/data
>>   backend:
>>     image: python:3.11-slim
>>     container_name: backend
>>     networks:
>>       - app-net
>>     ports:
>>       - "8000:8000"
>>     environment:
>>       - DB_HOST=postgres-db
>>       - DB_USER=admin
>>       - DB_PASS=secret
>>     depends_on:
>>       - postgres-db
>>
>> networks:
>>   app-net:
>>
>> volumes:
>>   pgdata:
>> '@ | Set-Content -Path docker-compose.yml

```

```

>> # Run and Verify
>> docker compose up -d

Directory: F:\DevOps Theory Lab\DockerLabs\lab-6

Mode                LastWriteTime         Length Name
----                -
d-----            01-05-2026  11:19 PM          Task3_P2
time="2026-05-01T23:19:44+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Task3_P2\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 10/10
✓ Image python:3.11-slim    Pulled                        8.1s
✓ Network task3_p2_app-net  Created                   0.1s
✓ Volume task3_p2_pgdata    Created                   0.0s
✓ Container postgres-db     Created                   0.2s
✓ Container backend         Created                   0.1s

PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task3_P2>

```

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> mkdir Task5; cd Task5
>>
>> # Create app.js
>> Set-Content -Path app.js -Value @"
>> const http = require('http');
>> http.createServer((req, res) => {
>>   res.end("Docker Compose Build Lab");
>> }).listen(3000);
>> "@
>>
>> # Create Dockerfile
>> @"
>> FROM node:18-alpine
>> WORKDIR /app
>> COPY app.js .
>> EXPOSE 3000
>> CMD ["node", "app.js"]
>> '@ | Set-Content -Path Dockerfile
>>
>> # Create docker-compose.yml
>> @"
>> version: '3.8'
>> services:
>>   nodeapp:
>>     build:
>>       context: .
>>       dockerfile: Dockerfile
>>     container_name: custom-node-app
>>     ports:
>>       - "3000:3000"
>> '@ | Set-Content -Path docker-compose.yml
>>
>> # Build and Run
>> docker compose up --build -d

```

```

#5 [internal] load build context
#5 transferring context: 154B done
#5 DONE 0.0s

#6 [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
#6 resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 0.0s done
#6 DONE 0.1s

#7 [2/3] WORKDIR /app
#7 DONE 0.0s

#8 [3/3] COPY app.js .
#8 DONE 0.0s

#9 exporting to image
#9 exporting layers 0.1s done
#9 exporting manifest sha256:c4f79d6bf5c755d7cded47fa4a0963f58ddf270ec3f02ae68a97cf32c549f2d6 done
#9 exporting config sha256:c9536ade069ece47bf3f9bd4a10d530f3076aed130f06027ad03089429cc5ccd 0.0s done
#9 exporting attestation manifest sha256:5730998565a246fcbaba92c2b0a0323acc55eabab6c3e8fe769f8d680b798219 0.0s done
#9 exporting manifest list sha256:7550bc4fd05baa2b27b6ab0198ce49869b78d9d34c321ed7093ce611b708d93a done
#9 naming to docker.io/library/task5-nodeapp:latest done
#9 unpacking to docker.io/library/task5-nodeapp:latest 0.0s done
#9 DONE 0.2s

#10 resolving provenance for metadata file
#10 DONE 0.0s
[+] up 3/3
✓ Image task5-nodeapp Built
✓ Network task5_default Created
✓ Container custom-node-app Created

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6> # Create directory and enter it

```

>> mkdir Task4; cd Task4
>> # Create docker-compose.yml with resource limits
>> @'
>> version: '3.8'
>> services:
>>   limited-app:
>>     image: nginx:alpine
>>     container_name: limited-app
>>     ports:
>>       - "9000:9000"
>>     restart: always
>>     deploy:
>>       resources:
>>         limits:
>>           cpus: '0.5'
>>           memory: 256M
>> '@ | Set-Content -Path docker-compose.yml
>> # Run and Verify
>> docker compose up -d

```

Directory: F:\DevOps Theory Lab\DockerLabs\lab-6

```

Mode                LastWriteTime         Length Name
----                -
d-----          01-05-2026  11:21 PM             Task4
time="2026-05-01T23:21:51+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Task4\\docker-compose.yml: the attribut
e `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✓ Network task4_default Created                                0.1s
✓ Container limited-app Created                                0.1s

```

PS F:\DevOps Theory Lab\DockerLabs\lab-6\Task4> |

8.6 EXPERIMENT 6 B: Multi-Container Application involving Docker Swarm

8.6.1 1. Running with Docker Swarm

Step 1: Initialize Swarm

```
docker swarm init
```

```

>> # Create the multi-container compose file
>> @'
>> version: '3.9'
>> services:
>>   db:
>>     image: mysql:5.7
>>     container_name: wordpress_db
>>     restart: always
>>     environment:
>>       MYSQL_ROOT_PASSWORD: rootpass
>>       MYSQL_DATABASE: wordpress
>>       MYSQL_USER: wpuser
>>       MYSQL_PASSWORD: wppass
>>     volumes:
>>       - db_data:/var/lib/mysql
>>   wordpress:
>>     image: wordpress:latest
>>     container_name: wordpress_app
>>     depends_on:
>>       - db
>>     ports:
>>       - "8080:80"
>>     restart: always
>>     environment:
>>       WORDPRESS_DB_HOST: db:3306
>>       WORDPRESS_DB_USER: wpuser
>>       WORDPRESS_DB_PASSWORD: wppass
>>       WORDPRESS_DB_NAME: wordpress
>>     volumes:
>>       - wp_data:/var/www/html
>> volumes:
>>   db_data:
>>   wp_data:
>> '@ | Set-Content -Path docker-compose.yml
>>
>> # 1. Start and Scale with Compose
>> docker compose up -d

```

Step 2: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
>> version: '3.9' ...
✓ Network exp6b_default Created 0.1s
✓ Volume exp6b_wp_data Created 0.0s
✓ Volume exp6b_db_data Created 0.0s
✓ Container wordpress_db Created 0.1s
✓ Container wordpress_app Created 0.2s
time="2026-05-01T23:28:31+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Exp6B\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 1/1
✓ Container wordpress_app Running 0.0s
WARNING: The "wordpress" service is using the custom container name "wordpress_app". Docker requires each container to have a unique name. Remove the custom name to scale the service

PS F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Exp6B> docker swarm init
Swarm initialized: current node (iy39cw2j7lh9f3dcnx4n2h7iv) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2nqt2xz0142j4m28gfnlevdm80zpeb5m4i5j9w83x90ec01mil-1r8sukfq844tu1m7brzyxjbyr 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
PS F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Exp6B> docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
```

Step 3: Scale Service

```
docker service scale wpstack_wordpress=3
```

```
PS F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Exp6B> docker stack services wpstack
>> docker service scale wpstack_wordpress=5
>>
ID                NAME                MODE                REPLICAS  IMAGE                PORTS
97nbx49unssp      wpstack_db          replicated          1/1        mysql:5.7
uf5c63uszwn       wpstack_wordpress    replicated          1/1        wordpress:latest     *:8080->80/tcp
wpstack_wordpress scaled to 5
overall progress: 5 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: running [=====>]
verify: Service wpstack_wordpress converged
PS F:\\DevOps Theory Lab\\DockerLabs\\lab-6\\Exp6B>
```

Figure 53: Docker Swarm Scaling

8.6.1.1 Comparison: Docker Compose vs Docker Swarm

Metric	Docker Compose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Manual	Built-in
Load Balancing	No	Yes (Internal)
Self-healing	No	Yes

8.7 Key Learning Outcomes

- **Docker Compose** is ideal for local development, testing, and learning.
 - **Docker Swarm** is useful for simple production clusters and easy scaling.
 - Multi-container applications require orchestration for management.
 - Internal networking and volumes are critical for persistence and communication.
 - Declarative configuration (YAML) is preferred for complex deployments.
-

8.8 Additional Resources

- [Docker Compose Overview](#)
- [Compose File Specification](#)
- [Docker Swarm Mode Introduction](#)
- [WordPress Docker Official Image](#)

9 Experiment 7: Jenkins, GitHub, and Docker Hub

9.1 Table of Contents

1. [Aim](#)
 2. [Objectives](#)
 3. [Theory](#)
 4. [Part A: GitHub Repository Setup](#)
 5. [Part B: Jenkins Setup using Docker](#)
 6. [Part C: Jenkins Configuration](#)
 7. [Part D: GitHub Webhook Integration](#)
 8. [Part E: Understanding Jenkins Pipeline Syntax](#)
 9. [Observations & Results](#)
 10. [Additional Resources](#)
-

9.2 1. Aim

To design and implement a complete **CI/CD pipeline** using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

9.3 2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool).
 - Create a structured GitHub repository with application and a Jenkinsfile.
 - Build Docker images from source code automatically.
 - Securely store Docker Hub credentials in Jenkins.
 - Automate the build and push process using **GitHub Webhooks**.
-

9.4 3. Theory

- **Jenkins:** A web-based GUI automation server used to build, test, and deploy software.
 - **CI/CD:**
 - **Continuous Integration (CI):** Code is automatically built and tested after each commit.
 - **Continuous Deployment (CD):** Built artifacts are automatically delivered.
 - **Workflow:** Developer → GitHub → Webhook → Jenkins → Build → Docker Hub.
-

9.5 Part A: GitHub Repository Setup

9.5.1 5.1 Project Structure

Create a repository named my-app with the following structure:

```
my-app/  
├─ app.py  
├─ requirements.txt  
├─ Dockerfile  
└─ Jenkinsfile
```

```

PS F:\DevOps Theory Lab> cd DockerLabs
PS F:\DevOps Theory Lab\DockerLabs> # 1. Create a workspace for Experiment 7
>> mkdir Exp7_Jenkins; cd Exp7_Jenkins
>>
>> # 2. Create the application file (Node.js)
>> Set-Content -Path index.js -Value @"
>> console.log('CI/CD Pipeline successful!');
>> console.log('App version: 1.0.0');
>> "@
>>
>> # 3. Create the Dockerfile
>> @"
>> FROM node:alpine
>> WORKDIR /app
>> COPY index.js .
>> CMD ["node", "index.js"]
>> '@ | Set-Content -Path Dockerfile

```

Directory: F:\DevOps Theory Lab\DockerLabs

Mode	LastWriteTime	Length	Name
d-----	01-05-2026 11:40 PM		Exp7_Jenkins

```

PS F:\DevOps Theory Lab\DockerLabs\Exp7_Jenkins> █

```

Figure 54: GitHub Repository Structure

9.5.2 5.3 Jenkinsfile (Pipeline Definition)

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = "your-dockerhub-username/myapp"
  }

  stages {
    stage('Clone Source') {
      steps {
        git 'https://github.com/your-username/my-app.git'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME:latest .'
      }
    }

    stage('Login to Docker Hub') {
      steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
          sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
        }
      }
    }

    stage('Push to Docker Hub') {
      steps {
        sh 'docker push $IMAGE_NAME:latest'
      }
    }
  }
}
```

9.6 Part B: Jenkins Setup using Docker

9.6.1 6.1 Create Docker Compose File

```
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
```

jenkins_home:

9.7 Part E: Understanding Jenkins Pipeline Syntax

9.7.1 Key Terms

- **pipeline:** The root block containing the entire definition.
 - **agent any:** Tells Jenkins to run this pipeline on any available node.
 - **stages:** Groups the different phases (Clone, Build, Auth, Push).
 - **steps:** The actual commands (e.g., sh, git).
 - **withCredentials:** A secure way to inject sensitive data into the environment temporarily.
-

9.8 Observations & Results

- **Automation:** Once configured, a simple git push triggers the entire build and push process.
- **Security:** Credentials are managed centrally in Jenkins and never exposed in the source code.

```
PS F:\DevOps Theory Lab\DockerLabs\Exp7_Jenkins> # 1. Create docker-compose.yml for Jenkins
>> @'
>> version: '3.8'
>> services:
>>   jenkins:
>>     image: jenkins/jenkins:lts
>>     container_name: jenkins-server
>>     privileged: true
>>     user: root
>>     ports:
>>       - "8080:8080"
>>       - "50000:50000"
>>     volumes:
>>       - jenkins_home:/var/jenkins_home
>>       - /var/run/docker.sock:/var/run/docker.sock
>> volumes:
>>   jenkins_home:
>> '@ | Set-Content -Path docker-compose-jenkins.yml
>>
>> # 2. Start Jenkins
>> docker compose -f docker-compose-jenkins.yml up -d
>>
>> # 3. Retrieve the Initial Admin Password (Take Screenshot)
>> docker exec jenkins-server cat /var/jenkins_home/secrets/initialAdminPassword
time="2026-05-01T23:51:36+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\Exp7_Jenkins\\docker-compose-jenkins.yml: the
attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 17/17
✓ Image jenkins/jenkins:lts          Pulled                28.3s
✓ Network exp7_jenkins_default       Created             0.1s
✓ Volume exp7_jenkins_jenkins_home   Created             0.0s
✓ Container jenkins-server           Created             1.4s
cat: /var/jenkins_home/secrets/initialAdminPassword: No such file or directory
PS F:\DevOps Theory Lab\DockerLabs\Exp7_Jenkins> |
```

Figure 55: Jenkins Pipeline Success

9.9 Additional Resources

- [Jenkins Documentation](#)
- [Using a Jenkinsfile](#)
- [Pipeline Syntax Guide](#)
- [GitHub Webhooks Guide](#)

10 Experiment 9: Infrastructure Automation using Ansible

10.1 Table of Contents

1. [Theory](#)
 2. [PART A – PRACTICAL TASK](#)
 3. [Ansible Demo with Docker Containers](#)
 4. [Multi-Container Ansible Exercise](#)
 5. [The Need for Ansible](#)
 6. [Conclusion](#)
 7. [Additional Resources](#)
-

10.2 1. Theory

10.2.1 What is Ansible?

Ansible is an open-source automation tool for configuration management, application deployment, and orchestration.

- **Agentless Architecture:** Uses SSH for Linux.
 - **Idempotency:** Predictable system states.
 - **Declarative Syntax:** Describes the desired state.
-

10.3 PART A – PRACTICAL TASK

10.3.1 2. Ansible Installation on Windows (via WSL)

10.3.1.1 Step 1: Install Ansible

```
sudo apt update
sudo apt install ansible -y
```

10.3.1.2 Step 2: Verify Installation

```
ansible --version
ansible localhost -m ping
```

10.4 3. Ansible Demo with Docker Containers

10.4.1 Step 1: Create SSH Key Pair in WSL

```
ssh-keygen -t rsa -b 4096
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

```

PS F:\DevOps Theory Lab\DockerLabs> # Open WSL and run these (or run via powershell wsl command)
>> wsl sudo apt update -y
>> wsl sudo apt install ansible -y
>> wsl ansible --version
>> ...
Setting up python3-resolvelib (1.0.1-1) ...
Setting up python3-kerberos (1.1.14-3.1build9) ...
Setting up python3-simplejson (3.19.2-1build2) ...
Setting up python3-xmltodict (0.13.0-1ubuntu0.24.04.1) ...
Setting up python3-packaging (24.0-1) ...
Setting up python3-jmespath (1.0.1-1) ...
Setting up ieee-data (20220827.1) ...
Setting up python3-dnspython (2.6.1-1ubuntu1) ...
Setting up python3-selinux (3.5-2ubuntu2.1) ...
Setting up python3-argcomplete (3.1.4-1ubuntu0.1) ...
Setting up python3-requests-ntlm (1.1.0-3) ...
Setting up python3-libcloud (3.4.1-5) ...
Setting up python3-netaddr (0.8.0-2ubuntu1) ...
Setting up python3-winrm (0.4.3-2) ...
Setting up ansible-core (2.16.3-0ubuntu2) ...
Setting up ansible (9.2.0+dfsg-0ubuntu5) ...
Processing triggers for man-db (2.12.0-4build2) ...
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
PS F:\DevOps Theory Lab\DockerLabs>

```

Figure 56: Ansible Installation and Ping Test

```

>> wsl ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ''
>>
>> # 3. Copy keys to current Windows directory for Docker build
>> wsl cp ~/.ssh/id_rsa.pub .
>> wsl cp ~/.ssh/id_rsa .
>>
>> # 4. Create the Dockerfile for the SSH Server
>> @'
>> FROM ubuntu
>> RUN apt update -y && apt install -y python3 python3-pip openssh-server
>> RUN mkdir -p /var/run/sshd
>> RUN echo 'root:password' | chpasswd && \
>>     sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
>>     sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
>>     sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config
>> RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh
>> COPY id_rsa /root/.ssh/id_rsa
>> COPY id_rsa.pub /root/.ssh/authorized_keys
>> RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys
>> RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd
>> EXPOSE 22
>> CMD ["/usr/sbin/sshd", "-D"]
>> '@ | Set-Content -Path Dockerfile
>>
>> # 5. Build the Ubuntu-Server image
>> docker build -t ubuntu-server .

```

Directory: F:\DevOps Theory Lab\DockerLabs

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	02-05-2026 03:16 AM		Exp9_Ansible

Generating public/private rsa key pair.
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub

```

|=. * o o S |
|.o * + |
|. o . o |
| o |
|. |
+----[SHA256]-----+
[+] Building 235.0s (14/14) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                  0.0s
=> => transferring dockerfile: 807B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [1/9] FROM docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c 0.0s
=> => resolve docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 4.20kB                                    0.0s
=> [2/9] RUN apt update -y && apt install -y python3 python3-pip openssh-server 207.4s
=> [3/9] RUN mkdir -p /var/run/ssh                                     0.3s
=> [4/9] RUN echo 'root:password' | chpasswd && sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/s 0.4s
=> [5/9] RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh             0.5s
=> [6/9] COPY id_rsa /root/.ssh/id_rsa                               0.0s
=> [7/9] COPY id_rsa.pub /root/.ssh/authorized_keys                 0.0s
=> [8/9] RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys 0.5s
=> [9/9] RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/ssh 0.4s
=> exporting to image                                              24.3s
=> => exporting layers                                              20.3s
=> => exporting manifest sha256:b2a5dba81a6ee887b82d2e8200abda2032e94ea78f2ed9b60da9b7f96a81daac 0.0s
=> => exporting config sha256:da49df93c4fbba943c0f66ea1072ccad8bebd26f2007e9c1c084ea2d7483458d 0.0s
=> => exporting attestation manifest sha256:f86be92d725ce7f0fcab26ab8635b8d3675dcf99db2ccb53e223bbbf44f869 0.0s
=> => exporting manifest list sha256:622abe6ce5280e0355a74d0849d2986add0c0cb8149ef371e41a6eede2eea02a 0.0s
=> => naming to docker.io/library/ubuntu-server:latest            0.0s
=> => unpacking to docker.io/library/ubuntu-server:latest          3.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vfr7axik52d3g228b6yx8e34t

PS F:\DevOps Theory Lab\DockerLabs\Exp9_Ansible>

```

10.4.2 Step 2: Create Dockerfile for SSH Server

```

FROM ubuntu
RUN apt update -y && apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/ssh
RUN echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/ssh_config
RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys
RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]

```

10.4.3 Step 3: Build and Run

```

docker build -t ubuntu-server .
docker run -d -p 2222:22 --name ssh-test-server ubuntu-server

```

10.5 4. Multi-Container Ansible Exercise

10.5.1 Step 1: Start 4 Test Servers

```

for i in {1..4}; do
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
done

```



```

PS F:\DevOps Theory Lab\DockerLabs\Exp9_Ansible> # 1. Start 4 Target Containers
>> for ($i=1; $i -le 4; $i++) {
>>     docker run -d --rm -p "220${$i}:22" --name "server${$i}" ubuntu-server
>> }
>>
>> # 2. Generate the inventory.ini file with Container IPs
>> " [servers]" | Set-Content -Path inventory.ini
>> for ($i=1; $i -le 4; $i++) {
>>     $ip = (docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' "server${$i}")
>>     "$ip" | Add-Content -Path inventory.ini
>> }
>>
>> # 3. Add Connection Variables to Inventory
>> '@'
>>
>> [servers:vars]
>> ansible_user=root
>> ansible_ssh_private_key_file=~/.ssh/id_rsa
>> ansible_python_interpreter=/usr/bin/python3
>> '@' | Add-Content -Path inventory.ini
>>
2fa046046a8451d87acac40e16d634c3fa42804411be6dd7ebe1655ed2e87adb
809a307cef68aeed8ab254bf01b6976538de1ce784ecf3b9371d74af762c9824
c9ac4aa78ae9fc95d98a507eabd716e6a5abbdac22d0b72c5e685d7e95217f1c
edf91da199888177f4bc78f5f9624531a6b8dc2f286f5fc896182d512c69f722
PS F:\DevOps Theory Lab\DockerLabs\Exp9_Ansible>

```

Figure 57: Docker SSH Server Running

10.5.2 Step 2: Run Playbook (playbook.yml)

```

---
- name: Configure multiple servers
  hosts: servers
  become: yes
  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
    - name: Create a custom test file
      copy:
        dest: /root/ansible_result.txt
        content: "Provisioned by Ansible"

```

Execute: `ansible-playbook -i inventory.ini playbook.yml`

```

PS F:\DevOps Theory Lab\dockerLabs\Exp9_Ansible> # Create inventory using localhost and mapped ports
>> @'
>> [servers]
>> server1 ansible_host=localhost ansible_port=2201
>> server2 ansible_host=localhost ansible_port=2202
>> server3 ansible_host=localhost ansible_port=2203
>> server4 ansible_host=localhost ansible_port=2204
>>
>> [servers:vars]
>> ansible_user=root
>> # Ensure we tell SSH to ignore host key checking for localhost ports
>> ansible_ssh_extra_args='-o StrictHostKeyChecking=no'
>> ansible_ssh_private_key_file=~/.ssh/id_rsa
>> ansible_python_interpreter=/usr/bin/python3
>> '@ | Set-Content -Path inventory.ini
PS F:\DevOps Theory Lab\dockerLabs\Exp9_Ansible> wsl ansible-playbook -i inventory.ini playbook.yml

PLAY [Configure multiple servers] *****

TASK [Gathering Facts] *****
ok: [server4]
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Update apt cache] *****
ok: [server4]
ok: [server2]
ok: [server3]
ok: [server1]

TASK [Install Python 3] *****
ok: [server2]
ok: [server3]
ok: [server4]
ok: [server1]

PS F:\DevOps Theory Lab\dockerLabs\Exp9_Ansible> wsl ansible-playbook -i inventory.ini playbook.yml
changed: [server1]
changed: [server4]
changed: [server3]
changed: [server2]

TASK [Display system information] *****
changed: [server4]
changed: [server1]
changed: [server3]
changed: [server2]

TASK [Print results] *****
ok: [server1] => {
  "msg": "Server Info: Linux 2fa046046a84 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux"
}
ok: [server2] => {
  "msg": "Server Info: Linux 809a307cef68 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux"
}
ok: [server3] => {
  "msg": "Server Info: Linux c9ac4aa78ae9 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux"
}
ok: [server4] => {
  "msg": "Server Info: Linux edf91da19988 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux"
}

PLAY RECAP *****
server1      : ok=6    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2      : ok=6    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server3      : ok=6    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server4      : ok=6    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

10.6 5. The Need for Ansible

- **Scalability:** Manage hundreds of servers at once.
 - **Consistency:** Identical configurations.
 - **Efficiency:** Automates repetitive tasks.
 - **IaC:** Version-controlled configuration.
-

10.7 Conclusion

This experiment demonstrated the power of Ansible for infrastructure automation, covering agentless configuration, inventory management, and declarative YAML playbooks.

10.8 Additional Resources

- [Ansible Documentation](#)
- [Ansible Galaxy](#)
- [Ansible Playbook Guide](#)
- [SSH Key Management Best Practices](#)

11 Experiment 10: SonarQube — Static Code Analysis

11.1 Table of Contents

1. [Theory](#)
 2. [Lab Architecture](#)
 3. [Hands-on Lab](#)
 4. [Integration & Best Practices](#)
 5. [Conclusion](#)
 6. [Additional Resources](#)
-

11.2 1. Theory

11.2.1 What is SonarQube?

SonarQube is an open-source platform that automatically scans source code for **bugs**, **security vulnerabilities**, and **maintainability issues**. This is known as **Static Application Security Testing (SAST)**.

11.2.2 Key Concepts

Term	Description
Quality Gate	Rules that code must pass before deployment.
Bug	Code likely to break at runtime.
Vulnerability	A security weakness.
Code Smell	Poorly written code that is hard to maintain.
Technical Debt	Time required to fix all current issues.

11.3 2. Lab Architecture

1. **SonarQube Server (“The Brain”)**: A web app that receives reports and applies rules.
2. **Sonar Scanner (“The Worker”)**: A CLI tool that reads code and sends reports to the server.

Flow: Your Code → Sonar Scanner → SonarQube Server → Dashboard.

11.4 3. Hands-on Lab

11.4.1 Step 1: Start the SonarQube Server

Create a docker-compose.yml file:

```
version: '3.8'
services:
  sonar-db:
    image: postgres:13
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
  sonarqube:
    image: sonarqube:lts-community
    ports:
      - "9000:9000"
```

```
depends_on:
  - sonar-db
```

Start: docker-compose up -d

```
PS F:\DevOps Theory Lab\DockeLabs> # 1. Create a workspace
>> mkdir Exp10_SonarQube; cd Exp10_SonarQube
>>
>> # 2. Create the docker-compose.yml for SonarQube + PostgreSQL
>> @'
>> version: '3.8'
>> services:
>>   sonar-db:
>>     image: postgres:13
>>     container_name: sonar-db
>>     environment:
>>       POSTGRES_USER: sonar
>>       POSTGRES_PASSWORD: sonar
>>       POSTGRES_DB: sonarqube
>>       POSTGRES_HOST_AUTH_METHOD: trust
>>     volumes:
>>       - sonar-db-data:/var/lib/postgresql/data
>>     networks:
>>       - sonarqube-lab
>>   sonarqube:
>>     image: sonarqube:lts-community
>>     container_name: sonarqube
>>     ports:
>>       - "9000:9000"
>>     environment:
>>       SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
>>       SONAR_JDBC_USERNAME: sonar
>>       SONAR_JDBC_PASSWORD: sonar
>>     volumes:
>>       - sonar-data:/opt/sonarqube/data
>>       - sonar-extensions:/opt/sonarqube/extensions
>>     depends_on:
>>       - sonar-db
>>     networks:
>>       - sonarqube-lab
>>   volumes:
>>     sonar-db-data:
```

```

PS F:\DevOps Theory Lab\DockerLabs> # 1. Create a workspace
>> mkdir Exp10_SonarQube; cd Exp10_SonarQube
>>
>> # 2. Create the docker-compose.yml for SonarQube + PostgreSQL
>> @' ...
>>   sonar-data:
>>   sonar-extensions:
>> networks:
>>   sonarqube-lab:
>>     driver: bridge
>> '@ | Set-Content -Path docker-compose.yml
>>
>> # 3. Start the server
>> docker compose up -d
>>
>> # 4. Wait for the server (Open http://localhost:9000 and log in as admin/admin)
>>

Directory: F:\DevOps Theory Lab\DockerLabs

Mode                LastWriteTime         Length Name
----                -
d-----          02-05-2026   03:31 AM                Exp10_SonarQube
time="2026-05-02T03:31:07+05:30" level=warning msg="F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 34/34
✓ Image postgres:13                                Pulled                                224.3s
✓ Image sonarqube:lts-community                      Pulled                                307.7s
✓ Network exp10_sonarqube_sonarqube-lab              Created                                0.1s
✓ Volume exp10_sonarqube_sonar-db-data               Created                                0.0s
✓ Volume exp10_sonarqube_sonar-data                  Created                                0.0s
✓ Volume exp10_sonarqube_sonar-extensions            Created                                0.0s
✓ Container sonar-db                                 Created                                0.8s
✓ Container sonarqube                                Created                                0.1s

```

11.4.2 Step 2: Generate Token

Access the SonarQube UI to generate an analysis token.

```
PS F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube> # 1. Create the project folder structure
>> mkdir -p sample-java-app/src/main/java/com/example
>> cd sample-java-app
>>
>> # 2. Create the Calculator class (with intentional code issues)
>> @'
>> package com.example;
>>
>> public class Calculator {
>>     // BUG: Division by zero risk
>>     public int divide(int a, int b) {
>>         return a / b;
>>     }
>>
>>     // CODE SMELL: Unused variable
>>     public int add(int a, int b) {
>>         int result = a + b;
>>         int unused = 100;
>>         return result;
>>     }
>>
>>     // VULNERABILITY: SQL Injection risk
>>     public String getUser(String userId) {
>>         String query = "SELECT * FROM users WHERE id = " + userId;
>>         return query;
>>     }
>>
>>     // CODE SMELL: Duplicated code
>>     public int multiply(int a, int b) {
>>         int result = 0;
>>         for (int i = 0; i < b; i++) { result += a; }
>>         return result;
>>     }
>> }
>> '@ | Set-Content -Path src/main/java/com/example/Calculator.java
>>
```

```
>> @'
>> <project>
>>   <modelVersion>4.0.0</modelVersion>
>>   <groupId>com.example</groupId>
>>   <artifactId>sample-app</artifactId>
>>   <version>1.0-SNAPSHOT</version>
>>   <properties>
>>     <maven.compiler.source>11</maven.compiler.source>
>>     <maven.compiler.target>11</maven.compiler.target>
>>     <sonar.projectKey>sample-java-app</sonar.projectKey>
>>     <sonar.host.url>http://localhost:9000</sonar.host.url>
>>   </properties>
>>   <build>
>>     <plugins>
>>       <plugin>
>>         <groupId>org.sonarsource.scanner.maven</groupId>
>>         <artifactId>sonar-maven-plugin</artifactId>
>>         <version>3.9.1.2184</version>
>>       </plugin>
>>     </plugins>
>>   </build>
>> </project>
>> '@ | Set-Content -Path pom.xml
>>
```

Directory: F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube\sample-java-app\src\main\java\com

Mode	LastWriteTime	Length	Name
d----	02-05-2026 03:37 AM		example

11.4.3 Step 2: Run the Scanner

Run analysis using the Maven plugin:

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

11.4.4 Step 3: View Results

Open <http://localhost:9000> to see the report.

```
PS F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube\sample-java-app> # Stop the SonarQube server and database
>> cd ..
>> docker compose down -v
time="2026-05-02T03:45:59+05:30" level=warning msg="F:\\DevOps Theory Lab\\DockerLabs\\Exp10_SonarQube\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 6/6
✓ Container sonarqube Removed 1.4s
✓ Container sonar-db Removed 0.5s
✓ Volume exp10_sonarqube_sonar-data Removed 0.0s
✓ Network exp10_sonarqube_sonarqube-lab Removed 0.7s
✓ Volume exp10_sonarqube_sonar-extensions Removed 0.1s
✓ Volume exp10_sonarqube_sonar-db-data Removed 0.1s
PS F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube>
```



```

PS F:\DevOps Theory Lab\DockerLabs\Exp10_SonarQube\sample-java-app> # Run the Sonar Scanner using Docker
>> docker run --rm `
>> --network sonarqube-lab `
>> -v "${PWD}:/usr/src" `
>> sonarsource/sonar-scanner-cli `
>> -Dsonar.host.url=http://sonarqube:9000 `
>> -Dsonar.projectKey=sample-java-app `
>> -Dsonar.login=YOUR_TOKEN
Unable to find image 'sonarsource/sonar-scanner-cli:latest' locally
latest: Pulling from sonarsource/sonar-scanner-cli
4f4fb700ef54: Pull complete
677b33267ef4: Download complete
b226ade5bac4: Downloading [=====>] 34.6MB/264.5MB
fd916ef360dd: Downloading [=====] 28.31MB/54.03MB
f95d770a13e7: Download complete
8f515facfbc5: Downloading [=====>] 23.07MB/170.2MB

```

Figure 58: Scanner Execution in Terminal

11.5 4. Integration & Best Practices

11.5.1 CI/CD Integration (Jenkins)

```

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh 'mvn clean verify sonar:sonar'
        }
    }
}

```

11.5.2 Best Practices

- **Scan Early:** Scan every Pull Request.
- **Enforce Quality Gates:** Block deployments if rules fail.
- **Zero Technical Debt:** Fix issues immediately.
- **Security:** Use secret managers for tokens.

11.6 Additional Resources

- [SonarQube Documentation](#)
- [SonarScanner CLI Guide](#)
- [OWASP Static Code Analysis Guide](#)
- [Clean Code Principles](#)

12 Experiment 11: Orchestration using Docker Swarm

12.1 Table of Contents

1. [Theory: Concept Continuation](#)
 2. [Practical Tasks](#)
 3. [Analysis: Compose vs Swarm](#)
 4. [Important Observations](#)
 5. [Additional Resources](#)
-

12.2 1. Theory: Concept Continuation

12.2.1 What is Orchestration?

Orchestration is the **automatic management of containers**. It handles:

- **Scaling**: Adjusting the number of running containers.
- **Self-healing**: Replacing failed containers.
- **Load balancing**: Distributing traffic.

12.2.2 The Progression Path

docker run → Docker Compose → Docker Swarm → Kubernetes

12.3 2. Practical Tasks

12.3.1 Task 1: Initialize Docker Swarm

Turn your machine into a **Manager Node**:

```
docker swarm init
```

Verify Swarm is active:

```
docker node ls
```

```
PS F:\DevOps Theory Lab\DockerLabs> # Create the directory for Lab 11
>> mkdir Exp11_Swarm
>> cd Exp11_Swarm
>>
>> # Copy the docker-compose.yml from Lab 6 (Task 2) to this folder ...
```

Mode	LastWriteTime	Length	Name
d-----	02-05-2026 01:02 PM		Exp11_Swarm

```
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> █
```

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> # Deploy the stack using the compose file (Name the stack: wpstack)
>> docker stack deploy -c docker-compose.yml wpstack
>>
>> # List the services to ensure they are starting
>> docker service ls
>>
>> # View the status of the specific WordPress service
>> docker service ps wpstack_wordpress
>>
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_mysql
Creating service wpstack_wordpress
ID            NAME                MODE                REPLICAS    IMAGE                PORTS
nbv5bl4fgywd  wpstack_mysql        replicated          1/1          mysql:5.7
3z4puxbc1zt6  wpstack_wordpress    replicated          0/1          wordpress:latest     *:8082->80/tcp
ID            NAME                IMAGE                NODE          DESIRED STATE    CURRENT STATE    ERROR
PORTS
kzu2tgivc057  wpstack_wordpress.1  wordpress:latest     docker-desktop Running          Assigned less than a second ago
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm>

```

12.3.2 Task 2: Deploy as a Stack

Deploy a group of services using a Compose file:

```
docker stack deploy -c docker-compose.yml wpstack
```

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> # Initialize Swarm Mode
>> docker swarm init
>>
>> # Verify that the node is active and is the Leader
>> docker node ls
Swarm initialized: current node (nasy26ypck7mevxi6ov33ja58) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0yig0f0j8v9adb7rdweuto6juxvq52r1e3d0yagyjpdcc44tefl-88ilmaq26my22e2ntaez08gu 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ID            HOSTNAME                STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
nasy26ypck7mevxi6ov33ja58 *  docker-desktop    Ready     Active           Leader             29.2.0
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm>

```

Figure 59: Stack Deployment

12.3.3 Task 3: Scale the Application

Scale WordPress to 3 replicas:

```
docker service scale wpstack_wordpress=3
```

Verification:

```
docker service ls          # Notice REPLICAS 3/3
```

12.3.4 Task 4: Test Self-Healing

1. Kill a container: `docker kill <container-id>`.
2. Swarm immediately starts a new one to maintain the desired state.

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> # Scale the WordPress service to 3 replicas
>> docker service scale wpstack_wordpress=3
>>
>> # Verify the scaling (REPLICAS should show 3/3)
>> docker service ls
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
nbv5b14fgywd	wpstack_mysql	replicated	1/1	mysql:5.7	
3z4puxbc1zt6	wpstack_wordpress	replicated	3/3	wordpress:latest	*:8082->80/tcp

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm>

```

Figure 60: Scaling Service

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> # 1. List running containers to find a WordPress container ID
>> docker ps
>>
>> # 2. Kill one of the WordPress containers (Replace <ID> with an actual ID from above)
>> docker kill <container_id>
>>
>> # 3. Watch Swarm immediately start a replacement container
>> docker service ps wpstack_wordpress
At line:5 char:13
+ docker kill <container_id>
+ ~
The '<' operator is reserved for future use.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : RedirectionNotSupported

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> docker kill 1c2d4b27f03e
1c2d4b27f03e
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> docker service ps wpstack_wordpress

```

ID	NAME	PORTS	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
t7b1cmgsv3y1	wpstack_wordpress.1		wordpress:latest	docker-desktop	Running	Running 6 seconds ago	
kzu2tgivc057	_ wpstack_wordpress.1		wordpress:latest	docker-desktop	Shutdown	Failed 12 seconds ago	"task: non-zero exit (137)"
it5u4bp4yr85	wpstack_wordpress.2		wordpress:latest	docker-desktop	Running	Running 2 minutes ago	
yr2uee99g25n	wpstack_wordpress.3		wordpress:latest	docker-desktop	Running	Running 2 minutes ago	

```

PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm>
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm> # Remove the stack (this stops all associated containers)
>> docker stack rm wpstack
>>
>> # Leave Swarm mode
>> docker swarm leave --force
Removing service wpstack_mysql
Removing service wpstack_wordpress
Removing network wpstack_default
Node left the swarm.
PS F:\DevOps Theory Lab\DockerLabs\Exp11_Swarm>

```

12.4 3. Analysis: Compose vs Swarm

Feature	Docker Compose	Docker Swarm
Scope	Single host	Multi-host cluster
Scaling	Manual	Declarative
Self-healing	Minimal	Built-in
Load Balancing	External needed	Built-in

12.5 4. Important Observations

- **Reuse:** The same Compose file works for both local dev and Swarm orchestration.
 - **Services:** You manage **Services**, while Swarm manages the **Containers**.
 - **IaC:** Swarm maintains the **desired state** automatically.
-

12.6 Additional Resources

- [Docker Swarm Mode Overview](#)
- [Docker Stack Deploy Guide](#)
- [High Availability in Docker Swarm](#)
- [Service Discovery and Load Balancing](#)

13 Experiment 12: Container Orchestration using Kubernetes

13.1 Table of Contents

1. [Objective](#)
 2. [Theory: Why Kubernetes?](#)
 3. [Core Concepts Mapping](#)
 4. [Hands-On Lab](#)
 5. [Summary of Commands](#)
 6. [Conclusion](#)
 7. [Additional Resources](#)
-

13.2 1. Objective

To understand why Kubernetes is used, learn its core concepts, and implement deployment, scaling, and self-healing through a hands-on lab using `kubectl`.

13.3 2. Theory: Why Kubernetes?

While Docker Swarm is simple, **Kubernetes (K8s)** is the industry standard for production environments due to its advanced feature set.

Reason	Explanation
Industry Standard Powerful Scheduling Cloud-Native	Most enterprises and cloud providers use Kubernetes. Automatically determines the best node to run your application. Native integration with AWS (EKS), Google Cloud (GKE), and Azure (AKS).

13.4 Core Concepts Mapping

Docker Concept	Kubernetes Equivalent	Description
Container	Pod	The smallest unit in K8s.
Compose Service	Deployment	Describes the desired state.
Load Balancing	Service	Exposes your app to the outside world.

13.5 3. Hands-On Lab

13.5.1 Task 1: Create a Deployment

A deployment defines which image to use and how many copies should run.

wordpress-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
```

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Apply: `kubectl apply -f wordpress-deployment.yaml`

13.5.2 Task 2: Expose the Deployment

wordpress-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30007
```

Apply: `kubectl apply -f wordpress-service.yaml`

13.5.3 Task 3: Scale the Deployment

`kubectl scale deployment wordpress --replicas=4`

13.5.4 Task 4: Test Self-Healing

Kubernetes automatically replaces deleted pods to maintain the desired state.

1. Delete a pod: `kubectl delete pod <pod-name>`
2. Watch Kubernetes recreate it: `kubectl get pods`

```

PS F:\DevOps Theory Lab\DockeRLabs\Exp12_K8s> '@'
>> apiVersion: apps/v1
>> kind: Deployment
>> metadata:
>>   name: wordpress
>> spec:
>>   replicas: 2
>>   selector:
>>     matchLabels:
>>       app: wordpress
>>   template:
>>     metadata:
>>       labels:
>>         app: wordpress
>>     spec:
>>       containers:
>>         - name: wordpress
>>           image: wordpress:latest
>>           ports:
>>             - containerPort: 80
>> '@ | Out-File -FilePath wordpress-deployment.yaml -Encoding utf8
PS F:\DevOps Theory Lab\DockeRLabs\Exp12_K8s> '@'
>> apiVersion: v1
>> kind: Service
>> metadata:
>>   name: wordpress-service
>> spec:
>>   type: NodePort
>>   selector:
>>     app: wordpress
>>   ports:
>>     - port: 80
>>       targetPort: 80
>>       nodePort: 30007
>> '@ | Out-File -FilePath wordpress-service.yaml -Encoding utf8
PS F:\DevOps Theory Lab\DockeRLabs\Exp12_K8s> █

```

Figure 61: Kubernetes Deployment


```

PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> # Set context to Docker Desktop (ensures you aren't pointing to a missing server)
>> kubectl config use-context docker-desktop
>>
>> # Apply the Deployment and Service
>> kubectl apply -f wordpress-deployment.yaml
>> kubectl apply -f wordpress-service.yaml
>>
>> # Check if pods are running (Status should change to 'Running')
>> kubectl get pods
Switched to context "docker-desktop".
deployment.apps/wordpress created
service/wordpress-service created
NAME                READY   STATUS             RESTARTS   AGE
wordpress-848c87c44f-hcnh6   0/1     ContainerCreating   0           0s
wordpress-848c87c44f-rsnc6   0/1     ContainerCreating   0           0s
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> # Check status again (Should show 'Running' and '1/1' Ready)
>> kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
wordpress-848c87c44f-hcnh6   1/1     Running   0           68s
wordpress-848c87c44f-rsnc6   1/1     Running   0           68s
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s>

```

Figure 62: Kubernetes Service

```

PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> # Scale to 4 replicas
>> kubectl scale deployment wordpress --replicas=4
>>
>> # Check the scaling progress
>> kubectl get pods
>>
>> # List the service to see the port mapping
>> kubectl get svc
deployment.apps/wordpress scaled
NAME                READY   STATUS             RESTARTS   AGE
wordpress-848c87c44f-72mbv   0/1     ContainerCreating   0           0s
wordpress-848c87c44f-hcnh6   1/1     Running             0           8m24s
wordpress-848c87c44f-jkvm7   0/1     ContainerCreating   0           0s
wordpress-848c87c44f-rsnc6   1/1     Running             0           8m24s
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP     10.96.0.1     <none>       443/TCP          12m
wordpress-service   NodePort      10.98.86.24   <none>       80:30007/TCP     8m24s
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s>

```

Figure 63: Kubernetes Scaling

```

PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> # Scale to 4 replicas
>> kubectl scale deployment wordpress --replicas=4
>>
>> # Check the scaling progress
>> kubectl get pods
>>
>> # List the service to see the port mapping
>> kubectl get svc
deployment.apps/wordpress scaled
NAME                                READY   STATUS              RESTARTS   AGE
wordpress-848c87c44f-72mbv         0/1     ContainerCreating   0           0s
wordpress-848c87c44f-hcnh6         1/1     Running             0           8m24s
wordpress-848c87c44f-jkvm7         0/1     ContainerCreating   0           0s
wordpress-848c87c44f-rsnc6         1/1     Running             0           8m24s
NAME                                TYPE               CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes                          ClusterIP          10.96.0.1    <none>         443/TCP          12m
wordpress-service                   NodePort           10.98.86.24  <none>         80:30007/TCP     8m24s
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> # 1. Get the name of one running pod
>> kubectl get pods
>>
>> # 2. Delete it (Replace <NAME> with an actual name like 'wordpress-769...')
>> kubectl delete pod wordpress-848c87c44f-hcnh6
>>
>> # 3. Verify Kubernetes immediately created a new replacement
>> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-848c87c44f-72mbv         1/1     Running   0           51s
wordpress-848c87c44f-hcnh6         1/1     Running   0           9m15s
wordpress-848c87c44f-jkvm7         1/1     Running   0           51s
wordpress-848c87c44f-rsnc6         1/1     Running   0           9m15s
pod "wordpress-848c87c44f-hcnh6" deleted from default namespace
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-848c87c44f-72mbv         1/1     Running   0           54s
wordpress-848c87c44f-jkvm7         1/1     Running   0           54s
wordpress-848c87c44f-rsnc6         1/1     Running   0           9m18s
wordpress-848c87c44f-xb8t4         0/1     ContainerCreating   0           3s
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s>
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s> kubectl delete service wordpress-service
>> kubectl delete deployment wordpress
service "wordpress-service" deleted from default namespace
deployment.apps "wordpress" deleted from default namespace
PS F:\DevOps Theory Lab\DockeLabs\Exp12_K8s>

```

13.6 5. Summary of Commands (Cheat Sheet)

Goal	Command
Apply configuration	<code>kubectl apply -f file.yaml</code>
List pods	<code>kubectl get pods</code>
List services	<code>kubectl get svc</code>
Scale replicas	<code>kubectl scale deployment <name> --replicas=N</code>

13.7 6. Conclusion

This experiment demonstrated the core functionality of Kubernetes: declarative deployments, stable service exposition, easy scaling, and automatic self-healing. Kubernetes provides the robustness required for enterprise-scale orchestration.

13.8 Additional Resources

- [Kubernetes Documentation](#)
- [Kubectl Cheat Sheet](#)
- [Interactive Kubernetes Tutorials](#)
- [Kubernetes Architecture Overview](#)